Assignment - Module 2

Meenu S

January 30, 2026

Introduction

This assignment explores some fundamental concepts of linear algebra that play a central role in both theoretical and applied mathematics. The focus is on inner product spaces, which provide a geometric structure to vector spaces and allow the definition of notions such as length, angle, and orthogonality. Building on this framework, the Gram–Schmidt orthonormalisation process is studied as a systematic method for constructing orthonormal bases from linearly independent sets of vectors.

The assignment also introduces the method of least squares regression, highlighting its interpretation in terms of orthogonal projections onto subspaces. Together, these topics illustrate how abstract linear algebraic ideas naturally lead to powerful computational techniques, bridging the gap between theory and applications.

1 Orthonormal Bases and Gram–Schmidt Orthonormalisation

1.1 Orthonormal Sets and Bases

Let V be an inner product space. A set of vectors $\{v_1, v_2, \ldots, v_n\}$ in V is said to be **orthonormal** if

$$\langle v_i, v_j \rangle = 0 \quad \text{for } i \neq j \quad \text{(orthogonality)} \tag{1}$$

$$\|v_i\| = 1 \quad \text{for all } i \quad \text{(normality)} \tag{2}$$

Equivalently,

$$\langle v_i, v_j \rangle = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

An orthonormal set that spans V is called an **orthonormal basis**.

1.2 Expansion in an Orthonormal Basis

If $\{u_1, u_2, \ldots, u_n\}$ is an orthonormal basis for V, then any vector $v \in V$ can be written as

$$v = \sum_{i=1}^{n} \langle v, u_i \rangle u_i$$

Let $\{v_1, v_2, \ldots, v_n\}$ be an orthonormal basis of an inner product space V. Then every vector $w \in V$ can be expressed as

$$w = \sum_{i=1}^{n} \langle w, v_i \rangle v_i.$$

Proof

Since $\{v_1, v_2, \ldots, v_n\}$ is a basis for V, any vector $w \in V$ can be written as a linear combination of these vectors:

$$w = \sum_{i=1}^{n} c_i v_i$$

for some scalars $c_1, c_2, \ldots, c_n$.

To determine the coefficients c_i, take the inner product of both sides with v_k for a fixed k:

$$\langle w, v_k \rangle = \left\langle \sum_{i=1}^{n} c_i v_i,\ v_k \right\rangle.$$

Using linearity of the inner product,

$$\langle w, v_k \rangle = \sum_{i=1}^{n} c_i \langle v_i, v_k \rangle.$$

Since the basis is orthonormal,

$$\langle v_i, v_k \rangle = \begin{cases} 1, & i = k, \\ 0, & i \neq k. \end{cases}$$

Hence all terms vanish except when $i = k$:

$$\langle w, v_k \rangle = c_k.$$

Thus each coefficient is given by

$$c_k = \langle w, v_k \rangle.$$

Substituting these values back into the expansion of w gives

$$w = \sum_{i=1}^{n} \langle w, v_i \rangle v_i.$$

Geometric Interpretation

Each term $\langle w, v_i \rangle v_i$ represents the projection of w onto the orthonormal direction v_i. Thus any vector can be written as the sum of its projections along mutually orthogonal unit directions.

Thus, the coefficients are given directly by inner products.

1.3 Gram–Schmidt Orthonormalisation

Given a linearly independent set $\{v_1, v_2, \ldots, v_n\}$ in an inner product space, the Gram–Schmidt process constructs an orthonormal set $\{u_1, u_2, \ldots, u_n\}$ that spans the same subspace.

Step 1: Construct Orthogonal Vectors

Define orthogonal vectors $w_1, w_2, \ldots, w_n$ as follows:

$$w_1 = v_1$$

$$w_2 = v_2 - \text{proj}_{w_1}(v_2)$$

$$w_3 = v_3 - \text{proj}_{w_1}(v_3) - \text{proj}_{w_2}(v_3)$$

In general,

$$w_k = v_k - \sum_{j=1}^{k-1} \text{proj}_{w_j}(v_k), \quad k = 2, 3, \ldots, n$$

where the projection is defined by

$$\text{proj}_{w_j}(v_k) = \frac{\langle v_k, w_j \rangle}{\langle w_j, w_j \rangle} w_j$$

The set $\{w_1, \ldots, w_n\}$ is mutually orthogonal.

Step 2: Normalisation

Define orthonormal vectors

$$u_i = \frac{w_i}{\|w_i\|}, \quad i = 1, 2, \ldots, n$$

Then $\{u_1, u_2, \ldots, u_n\}$ forms an orthonormal basis for

$$\text{span}\{v_1, v_2, \ldots, v_n\}$$

1.4 Key Property

Gram–Schmidt does not change the span:

$$\text{span}\{v_1, \ldots, v_n\} = \text{span}\{w_1, \ldots, w_n\} = \text{span}\{u_1, \ldots, u_n\}$$

It only converts the basis into an orthogonal and then orthonormal form.

```
import numpy as np
# from fractions import Fraction # No longer needed if not
   displaying fractions
import matplotlib.pyplot as plt11

# --- User Input Section ---
# Prompt the user to enter the dimension of the vector space.
# Input validation ensures a positive integer is entered.
while True:
    try:
        dimension = int(input("Enter the dimension of the vector
           space (a positive integer): "))
        if dimension <= 0:
            print("Dimension must be a positive integer. Please try
               again.")
        else:
            break
    except ValueError:
        print("Invalid input. Please enter an integer.")

print(f"The dimension of the vector space is: {dimension}")

# Prompt the user to enter the number of vectors they wish to
   orthonormalize.
# Input validation ensures a positive integer is entered.
while True:
    try:
        num_vectors = int(input("Enter the number of vectors you
           want to orthonormalize: "))
        if num_vectors <= 0:
            print("Number of vectors must be a positive integer.
               Please try again.")
        else:
            break
    except ValueError:
        print("Invalid input. Please enter an integer.")

print(f"You will enter {num_vectors} vectors.")

# --- Helper Functions ---
```

```
# Function to safely get numerical elements for a vector from user
   input.
def get_vector_elements(dimension, vector_name):
    elements = []
    print(f"\nEnter elements for {vector_name}:")
    for i in range(dimension):
        while True:
            try:
                element = float(input(f"Enter element {i+1} of
                   {dimension}: "))
                elements.append(element)
                break
            except ValueError:
                print("Invalid input. Please enter a number.")
    return np.array(elements)


# --- Vector Input Gathering ---
input_vectors = []
# Loop to gather all input vectors from the user.
for i in range(num_vectors):
    # The first vector is 'u1', subsequent ones are 'v2', 'v3', etc.
    if i == 0:
        vector_label = "u1"
    else:
        vector_label = f"v_{i+1}"

    # Get elements for the current vector from the user.
    vec = get_vector_elements(dimension, f"Vector {vector_label}")
    input_vectors.append(vec)

    # Display the created vector for confirmation, now in decimal
       format.
    print(f"\nCreated Vector {vector_label}:")
    print(f"[{', '.join(map(lambda x: f'{x:.8f}', vec))}]") #
       Display as decimals (8f)
    print(f"Shape of Vector {vector_label}: {vec.shape}")

# --- Gram-Schmidt orthonormalization Process ---
# List to store the orthogonal vectors (denoted as u_i in
   Gram-Schmidt).
orthogonal_vectors = []
# List to store the orthonormal basis vectors (denoted as e_i).
orthonormal_basis = []

print("\n--- Gram-Schmidt orthonormalization Process ---")
```

```
# Iterate through each of the user-provided input vectors (v_k).
for i, v_k in enumerate(input_vectors):
    # Initialize the current orthogonal vector u_k as a copy of the
       input v_k.
    # We will subtract projections from this copy.
    u_k = v_k.copy()

    # Subtract the projections of v_k onto all previously found
       orthogonal vectors (u_j).
    # This ensures that the resulting u_k is orthogonal to all u_j
       (j < i).
    for j in range(i):
        u_j = orthogonal_vectors[j] # Retrieve a previously
           orthogonalized vector.

        # Calculate the scalar projection factor of v_k onto u_j.
        # Formula: (v_k . u_j) / (u_j . u_j)
        magnitude_u_j_squared = np.dot(u_j, u_j)

        # Handle cases where a previous orthogonal vector might be
           a zero vector.
        # This can happen if the input vectors are linearly
           dependent.
        if magnitude_u_j_squared == 0:
            print(f"Warning: Orthogonal vector u_{j+1} is a zero
               vector. Skipping projection as it won't contribute.")
            continue # Skip projection if u_j is a zero vector to
               avoid division by zero.

        projection_factor = np.dot(v_k, u_j) / magnitude_u_j_squared

        # Subtract the projection of v_k onto u_j from u_k.
        # This removes the component of v_k that is parallel to u_j.
        u_k -= projection_factor * u_j

    # Add the newly computed orthogonal vector u_k to the list of
       orthogonal vectors.
    orthogonal_vectors.append(u_k)

    # Display the current orthogonal vector u_k and its magnitude.
    print(f"\nOrthogonal Vector u_{i+1}:")
    print(f"[{', '.join(map(lambda x: f'{x:.8f}', u_k))}]") #
       Display as decimals (8f)
    print(f"Magnitude of u_{i+1}: {np.linalg.norm(u_k):.8f}") #
       Increased decimal precision

    # Normalize u_k to obtain the orthonormal vector e_k.
```

```
    magnitude_u_k = np.linalg.norm(u_k)
    if magnitude_u_k == 0:
        # If u_k is a zero vector (e.g., if v_k was linearly
           dependent on previous vectors),
        # it cannot be normalized. We use a zero vector for e_k.
        print(f"Warning: Orthogonal vector u_{i+1} is a zero
           vector. It cannot be normalized. Using zero vector for
           e_{i+1}.")
        e_k = np.zeros(dimension)
    else:
        # Normalize u_k by dividing it by its Euclidean norm
           (magnitude).
        e_k = u_k / magnitude_u_k
    orthonormal_basis.append(e_k)

    # Display the current orthonormal vector e_k and its magnitude.
    print(f"Normalized Vector e_{i+1}:")
    print(f"[{', '.join(map(lambda x: f'{x:.8f}', e_k))}]") #
       Display as decimals (8f)
    print(f"Magnitude of e_{i+1}: {np.linalg.norm(e_k):.8f}") #
       Increased decimal precision


# --- Summary of Results ---

print("\n--- Summary of Orthogonal Vectors ---")
for i, u_vec in enumerate(orthogonal_vectors):
    print(f"u_{i+1}: [{', '.join(map(lambda x: f'{x:.8f}',
       u_vec))}]") # Display as decimals (8f)

print("\n--- Summary of Orthonormal Basis ---")
for i, e_vec in enumerate(orthonormal_basis):
    print(f"e_{i+1}: [{', '.join(map(lambda x: f'{x:.8f}',
       e_vec))}]") # Display as decimals (8f)

# --- Verification of Orthogonality and Normality ---

print("\n--- Verification ---")
# Check orthogonality for all unique pairs of orthonormal basis
   vectors.
# Their dot product should be approximately 0.
if num_vectors > 1:
    print("Checking orthogonality of orthonormal basis vectors (e_i
       dot e_j for i != j should be close to 0):")
    for i in range(len(orthonormal_basis)):
        for j in range(i + 1, len(orthonormal_basis)):
            dot_prod = np.dot(orthonormal_basis[i],
```

```
                orthonormal_basis[j])
            print(f"e_{i+1} . e_{j+1} = {dot_prod:.8f}") # Display
                dot product as decimals (8f).
            # A small tolerance (1e-9) is used to account for
                floating-point inaccuracies.
            if abs(dot_prod) > 1e-9:
                print(f"  (Warning: e_{i+1} and e_{j+1} are not
                    orthogonal)")

# Check normality for each orthonormal basis vector.
# Their magnitude should be approximately 1.
print("Checking normality of orthonormal basis vectors (magnitude
    should be close to 1):")
for i, e_vec in enumerate(orthonormal_basis):
    mag = np.linalg.norm(e_vec)
    print(f"|e_{i+1}| = {mag:.8f}") # Display magnitude as decimals
        (8f).
    # A small tolerance (1e-9) is used. Only warn if the original
        u_k was not a zero vector.
    if abs(mag - 1.0) > 1e-9 and
        np.linalg.norm(orthogonal_vectors[i]) != 0:
         print(f"  (Warning: e_{i+1} is not normal)")


# --- Plotting Section ---
print("\n--- Plotting Vectors ---")

ax = None # Initialize ax to None
plot_dim = 0 # Initialize plot_dim

if dimension == 2:
    plot_dim = 2
    plt.figure(figsize=(8, 8))
    ax = plt.gca() # Get current axes for 2D
    ax.axvline(0, color='gray', linewidth=0.5)
    ax.axhline(0, color='gray', linewidth=0.5)
    ax.grid(True, linestyle='--', alpha=0.6)
    ax.set_xlabel('X-axis')
    ax.set_ylabel('Y-axis')
    ax.set_title('Vector Visualization (2D)')

elif dimension == 3:
    from mpl_toolkits.mplot3d import Axes3D
    plot_dim = 3
    fig = plt.figure(figsize=(12, 12)) # Make figure slightly larger
    ax = fig.add_subplot(111, projection='3d')
    ax.grid(True, linestyle='--', alpha=0.6)
```

```
    ax.set_xlabel('X-axis')
    ax.set_ylabel('Y-axis')
    ax.set_zlabel('Z-axis')
    ax.set_title('Vector Visualization (3D Gram-Schmidt)')

    ax.view_init(elev=20, azim=120) # Reverted to default view

else:
    print("Plotting is currently supported for 2D and 3D vectors
       only.")
    print(f"Dimension of {dimension} is not supported for full
       visualization. No plot will be generated.")

if ax is not None:
    # Get a palette of distinct base colors for each original
       vector (v1, v2, v3, etc.)
    # 'tab10' provides 10 distinct categorical colors.
    base_colors_cmap = plt.colormaps.get_cmap('tab10')
    plane_color = 'skyblue' # Color for the span plane

    # Determine max_val for axis limits across all vectors
    all_coords = []
    for v_list in [input_vectors, orthogonal_vectors,
       orthonormal_basis]:
        for vec in v_list:
            if len(vec) >= plot_dim:
                all_coords.extend(np.abs(vec[:plot_dim]))

    if all_coords:
        max_coord_value = np.max(all_coords)
        max_val = max(max_coord_value, 1.5)
    else:
        max_val = 1.5

    # Plot Initial Vectors
    for i, vec in enumerate(input_vectors):
        if np.linalg.norm(vec[:plot_dim]) > 1e-9:
            current_base_color = base_colors_cmap(i % 10) # Assign
               a unique base color to each original vector's family
            if dimension == 2:
                ax.quiver(0, 0, vec[0], vec[1], angles='xy',
                   scale_units='xy', scale=1,
                   color=current_base_color, label=f'Initial
                   v_{i+1}', alpha=0.7, width=0.005)
            elif dimension == 3:
                ax.quiver(0, 0, 0, vec[0], vec[1], vec[2],
                   color=current_base_color, label=f'Initial
```

```
                v_{i+1}', alpha=0.7, arrow_length_ratio=0.1)

    # Plot Orthogonal Vectors (same base color, dashed line,
       thicker)
    for i, vec in enumerate(orthogonal_vectors):
        if np.linalg.norm(vec[:plot_dim]) > 1e-9:
            current_base_color = base_colors_cmap(i % 10) #
               Maintain the same base color for the family
            if dimension == 2:
                ax.quiver(0, 0, vec[0], vec[1], angles='xy',
                   scale_units='xy', scale=1,
                   color=current_base_color, label=f'Orthogonal
                   u_{i+1}', linestyle='--', alpha=0.9, width=0.008)
            elif dimension == 3:
                ax.quiver(0, 0, 0, vec[0], vec[1], vec[2],
                   color=current_base_color, label=f'Orthogonal
                   u_{i+1}', linestyle='--', alpha=0.9,
                   arrow_length_ratio=0.15)

    # Plot Orthonormal Vectors (same base color, dotted line,
       slightly thinner)
    for i, vec in enumerate(orthonormal_basis):
        if np.linalg.norm(vec[:plot_dim]) > 1e-9:
            current_base_color = base_colors_cmap(i % 10) #
               Maintain the same base color for the family
            if dimension == 2:
                ax.quiver(0, 0, vec[0], vec[1], angles='xy',
                   scale_units='xy', scale=1,
                   color=current_base_color, label=f'Orthonormal
                   e_{i+1}', linestyle=':', alpha=0.9, width=0.004)
            elif dimension == 3:
                ax.quiver(0, 0, 0, vec[0], vec[1], vec[2],
                   color=current_base_color, label=f'Orthonormal
                   e_{i+1}', linestyle=':', alpha=0.9,
                   arrow_length_ratio=0.08)

    # --- Plotting the Span (Plane) if applicable for 3D ---
    if dimension == 3:
        non_zero_orthonormal = [vec for vec in orthonormal_basis if
           np.linalg.norm(vec) > 1e-9]

        if len(non_zero_orthonormal) == 2:
            e1_span = non_zero_orthonormal[0]
            e2_span = non_zero_orthonormal[1]

            # Generate a grid for the plane
            s_vals = np.linspace(-1.5 * max_val, 1.5 * max_val, 20)
```

```
            t_vals = np.linspace(-1.5 * max_val, 1.5 * max_val, 20)
            S, T = np.meshgrid(s_vals, t_vals)

            # Calculate the coordinates of the points on the plane
            X_plane = S * e1_span[0] + T * e2_span[0]
            Y_plane = S * e1_span[1] + T * e2_span[1]
            Z_plane = S * e1_span[2] + T * e2_span[2]

            ax.plot_surface(X_plane, Y_plane, Z_plane,
                            alpha=0.2, color=plane_color,
                               rstride=1, cstride=1,
                               edgecolor='none')
            # Add a proxy artist for the legend entry for
               plot_surface
            ax.scatter([], [], [], color=plane_color, alpha=0.2,
               label='Span of e1, e2')
            print("\nNote: A plane representing the span of e1 and
               e2 has been plotted.")
        elif len(non_zero_orthonormal) < 2:
            print("\nNote: Fewer than two non-zero orthonormal
               vectors found. Cannot plot a plane representing span
               in 3D.")
        # If len(non_zero_orthonormal) > 2, it spans a 3D space,
           which is implicitly the whole plot

    ax.set_xlim([-max_val, max_val]) # Positive X to the right
    ax.set_ylim([-max_val, max_val]) # Positive Y upwards (standard)
    if dimension == 3:
        ax.set_zlim([-max_val, max_val])
        # Set equal aspect ratio for 3D plot
        ax.set_box_aspect([1,1,1]) # This sets the aspect ratio to
           be equal across all axes

    ax.legend()
    if dimension == 2:
        ax.set_aspect('equal', adjustable='box') # Only for 2D plots
    plt.tight_layout()
    plt.show()
```

Problem: Apply the Gram–Schmidt orthonormalization process to the vectors

$$v_1 = (0, 1, 0), \quad v_2 = (1, 1, 1), \quad v_3 = (1, 0, 1)$$

to obtain an orthonormal basis for their span in $\mathbb{R}^3$.

OUTPUT

Enter the dimension of the vector space (a positive integer): 3
The dimension of the vector space is: 3
Enter the number of vectors you want to orthonormalize: 3
You will enter 3 vectors.

Enter elements for Vector u1:
Enter element 1 of 3: 0
Enter element 2 of 3: 1
Enter element 3 of 3: 0

Created Vector u1:
[0.00000000, 1.00000000, 0.00000000]
Shape of Vector u1: (3,)

Enter elements for Vector v_2:
Enter element 1 of 3: 1
Enter element 2 of 3: 1
Enter element 3 of 3: 1

Created Vector v_2:
[1.00000000, 1.00000000, 1.00000000]
Shape of Vector v_2: (3,)

Enter elements for Vector v_3:
Enter element 1 of 3: 1
Enter element 2 of 3: 0
Enter element 3 of 3: 1

Created Vector v_3:
[1.00000000, 0.00000000, 1.00000000]
Shape of Vector v_3: (3,)

— Gram-Schmidt orthonormalization Process —

Orthogonal Vector u_1:
[0.00000000, 1.00000000, 0.00000000]
Magnitude of u_1: 1.00000000
Normalized Vector e_1:
[0.00000000, 1.00000000, 0.00000000]
Magnitude of e_1: 1.00000000

Orthogonal Vector u_2:
[1.00000000, 0.00000000, 1.00000000]

Magnitude of u_2: 1.41421356
Normalized Vector e_2:
[0.70710678, 0.00000000, 0.70710678]
Magnitude of e_2: 1.00000000

Orthogonal Vector u_3:
[0.00000000, 0.00000000, 0.00000000]
Magnitude of u_3: 0.00000000
Warning: Orthogonal vector u_3 is a zero vector. It cannot be normalized. Using zero vector for e_3.
Normalized Vector e_3:
[0.00000000, 0.00000000, 0.00000000]
Magnitude of e_3: 0.00000000

— Summary of Orthogonal Vectors —
u_1: [0.00000000, 1.00000000, 0.00000000]
u_2: [1.00000000, 0.00000000, 1.00000000]
u_3: [0.00000000, 0.00000000, 0.00000000]

— Summary of Orthonormal Basis —
e_1: [0.00000000, 1.00000000, 0.00000000]
e_2: [0.70710678, 0.00000000, 0.70710678]
e_3: [0.00000000, 0.00000000, 0.00000000]

— Verification —
Checking orthogonality of orthonormal basis vectors (e_i dot e_j for i != j should be close to 0):
e_1 . e_2 = 0.00000000
e_1 . e_3 = 0.00000000
e_2 . e_3 = 0.00000000
Checking normality of orthonormal basis vectors (magnitude should be close to 1):
—e_1— = 1.00000000
—e_2— = 1.00000000
—e_3— = 0.00000000

— Plotting Vectors —

(Note: A plane representing the span of e1 and e2 has been plotted
(Here,the span is only with the 2 vectors e1 and e2 as u3 is a zero vector.)

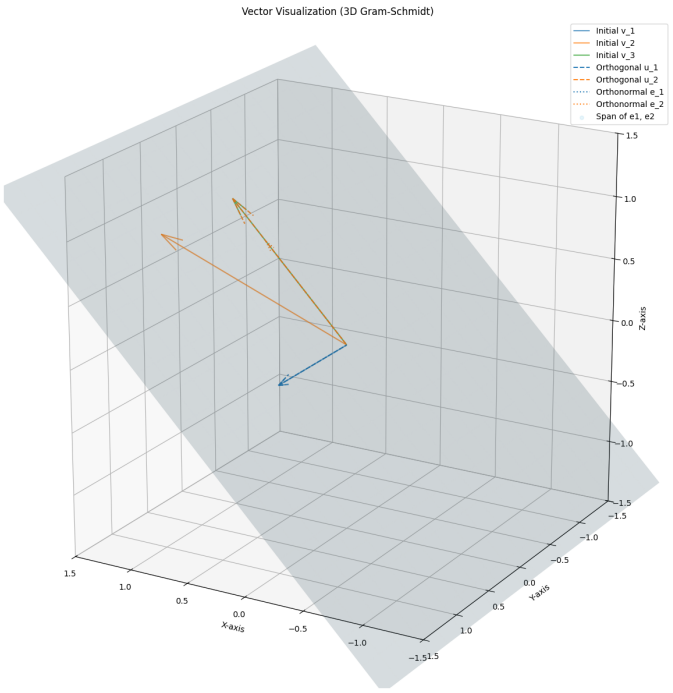

Figure 1: Orthonormalisation

The Gram–Schmidt process was implemented for functions using Python(below). Since numerical integration gives decimal approximations, the symbolic equivalents of the orthonormal functions are displayed.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad

print("Necessary libraries imported successfully.")

def functional_inner_product(f1, f2, interval):
    """Calculates the functional inner product of two functions
       over a given interval."""
    a, b = interval
    integrand = lambda x: f1(x) * f2(x)
    result, _ = quad(integrand, a, b)
    return result

def functional_norm(f, interval):
    """Calculates the functional norm of a single function over a
       given interval."""
    # The norm is the square root of the inner product of the
       function with itself.
    norm_squared = functional_inner_product(f, f, interval)
    # Ensure the result is non-negative before taking the square
       root due to potential floating point errors.
```

```
    return np.sqrt(max(0, norm_squared))

print("Functional inner product and norm helper functions defined.")

print("\n--- Input for Functional Gram-Schmidt ---")

# 1. Prompt the user for the integration interval [a, b]
while True:
    try:
        a = float(input("Enter the start point of the integration
           interval (a): "))
        break
    except ValueError:
        print("Invalid input. Please enter a numerical value for
           'a'.")

while True:
    try:
        b = float(input("Enter the end point of the integration
           interval (b): "))
        if b <= a:
            print("The end point 'b' must be greater than the start
               point 'a'. Please re-enter.")
        else:
            break
    except ValueError:
        print("Invalid input. Please enter a numerical value for
           'b'.")

interval = (a, b)
print(f"Integration interval set to: [{a}, {b}]")

# 2. Prompt the user for the number of functions
while True:
    try:
        num_functions = int(input("Enter the number of functions
           you want to orthonormalize: "))
        if num_functions <= 0:
            print("Number of functions must be a positive integer.
               Please try again.")
        else:
            break
    except ValueError:
        print("Invalid input. Please enter an integer.")

print(f"You will enter {num_functions} functions.")
```

```
# 3. Create empty lists to store function strings and callable
   lambda functions
input_function_strings = []
callable_functions = []

# 4. Loop to gather all input functions from the user
for i in range(num_functions):
    while True:
        function_str = input(f"Enter function f_{i+1} as a string
           (e.g., 'x', 'x**2', 'np.sin(x)'): ")
        try:
            # Attempt to create a lambda function. Use a dummy
               variable (e.g., test_x) to validate syntax.
            # We provide np in the scope for functions like
               np.sin(x)
            test_x = 0.5 # A value within a typical interval
               (adjust if needed)
            # Evaluate the function string with 'x' and 'np' in
               scope
            eval(f"lambda x: {function_str}", {"np": np})(test_x)

            # If successful, create the actual callable lambda
               function
            callable_func = eval(f"lambda x: {function_str}",
               {"np": np})
            callable_functions.append(callable_func)
            input_function_strings.append(function_str)
            print(f"Function f_{i+1} ('{function_str}')
               successfully added.")
            break # Exit the inner loop, move to the next function
        except (SyntaxError, NameError, TypeError) as e:
            print(f"Invalid function input or syntax error: {e}.
               Please re-enter.")
        except Exception as e:
            print(f"An unexpected error occurred: {e}. Please
               re-enter.")

print("\nAll functions have been entered and validated.")

print("\n--- Functional Gram-Schmidt Orthonormalization Process
   ---")

# 1. Initialize two empty lists
orthogonal_functions = []
orthonormal_functions = []

# 2. Iterate through each callable_func
```

```
for i, v_k in enumerate(callable_functions):
    # Display current input function being processed
    print(f"\nProcessing input function f_{i+1} (original:
        '{input_function_strings[i]}')")

    # 3a. Create a current function u_k (conceptually similar to
        v_k)
    # Initialize u_k such that u_k(x) = v_k(x)
    u_k_current = v_k # Start with v_k

    # 3b. For each previously found orthogonal function u_j
    for j in range(i):
        u_j = orthogonal_functions[j]

        # Calculate the scalar projection factor: (v_k . u_j) /
            (u_j . u_j)
        inner_product_u_j_u_j = functional_inner_product(u_j, u_j,
            interval)

        # 3b.ii. If inner_product_u_j_u_j is zero, skip projection
        if abs(inner_product_u_j_u_j) < 1e-9: # Check against a
            small tolerance
             print(f"  Warning: Orthogonal function u_{j+1} is
                approximately zero. Skipping projection of f_{i+1}
                onto u_{j+1}.")
             continue

        inner_product_v_k_u_j = functional_inner_product(v_k, u_j,
            interval)
        projection_factor = inner_product_v_k_u_j /
            inner_product_u_j_u_j

        # 3b.iii. Subtract the projection of v_k onto u_j from u_k
        # This requires creating a new lambda function for
            u_k_current
        # Store the previous u_k_current to use in the new lambda
            definition
        temp_u_k = u_k_current # Capture the current state of
            u_k_current
        u_k_current = lambda x, prev_u_k=temp_u_k,
            proj_factor=projection_factor, proj_func=u_j:
            prev_u_k(x) - proj_factor * proj_func(x)

        print(f"  Subtracted projection onto u_{j+1} (factor:
            {projection_factor:.4f})")

    # 3c. Append the resulting u_k to orthogonal_functions
```

```
    orthogonal_functions.append(u_k_current)
    print(f"  Orthogonal function u_{i+1} created.")

    # 3d. Calculate the norm of u_k
    norm_u_k = functional_norm(u_k_current, interval)

    # 3e. If the norm of u_k is zero, append a zero function and
       warn
    if abs(norm_u_k) < 1e-9:
        print(f"  Warning: Orthogonal function u_{i+1} is
           approximately zero. It cannot be normalized.")
        e_k = lambda x: 0.0 # Define a zero function
    # 3f. Otherwise, normalize u_k
    else:
        e_k = lambda x, func=u_k_current, norm=norm_u_k: func(x) /
           norm

    orthonormal_functions.append(e_k)
    print(f"  Orthonormal function e_{i+1} created (norm:
       {norm_u_k:.4f}).")

print("\nFunctional Gram-Schmidt process completed.")



# --- Symbolic Equivalents (for reference) ---
print("\n--- Symbolic Equivalents (for reference) ---")
print("For the input functions '1', 'x', 'x**2' over [-1, 1], the
   orthonormal functions correspond to scaled Legendre
   polynomials:")
print("e_1(x) ~= 1 / sqrt(2)")
print("e_2(x) ~= sqrt(3/2) * x")
print("e_3(x) ~= sqrt(5/8) * (3x^2 - 1)")

print("\n--- Verification of Orthogonality and Normality ---")

# 1. Check orthogonality for all unique pairs of orthonormal
   functions.
print("Checking orthogonality of orthonormal functions (inner
   product should be close to 0):")
for i in range(len(orthonormal_functions)):
    for j in range(i + 1, len(orthonormal_functions)):
        e_i = orthonormal_functions[i]
        e_j = orthonormal_functions[j]
        dot_prod = functional_inner_product(e_i, e_j, interval)
        print(f"  <e_{i+1}, e_{j+1}> = {dot_prod:.8f}")
        # A small tolerance (1e-6) is used to account for
```

```
            floating-point inaccuracies.
        if abs(dot_prod) > 1e-6:
            print(f"    (Warning: e_{i+1} and e_{j+1} are not
                orthogonal)")

# 2. Check normality for each orthonormal function.
print("\nChecking normality of orthonormal functions (norm should
    be close to 1):")
for i, e_k in enumerate(orthonormal_functions):
    mag = functional_norm(e_k, interval)
    print(f"  ||e_{i+1}|| = {mag:.8f}")
    # A small tolerance (1e-6) is used.
    if abs(mag - 1.0) > 1e-6:
        print(f"    (Warning: e_{i+1} is not normal)")

print("\n--- Plotting Functional Gram-Schmidt Results ---")

# 1. Generate an array of x values within the interval [a, b]
x_values = np.linspace(interval[0], interval[1], 500) # 500 points
    for smooth curves

plt.figure(figsize=(12, 8))

# Get a color map for distinct colors
colors_cmap = plt.colormaps.get_cmap('tab10')
colors = [colors_cmap(x) for x in np.linspace(0, 1,
    len(callable_functions))]

# 2. Evaluate and plot original functions
print("Plotting original functions...")
for i, func in enumerate(callable_functions):
    y_values = [func(x) for x in x_values]
    plt.plot(x_values, y_values, label=f'Original f_{i+1}:
        {input_function_strings[i]}', color=colors[i],
        linestyle='-', linewidth=2)

# 3. Evaluate and plot orthogonal functions
print("Plotting orthogonal functions...")
for i, func in enumerate(orthogonal_functions):
    y_values = [func(x) for x in x_values]
    # Check if the orthogonal function is essentially zero to avoid
        plotting a flat line that might obscure others
    if np.max(np.abs(y_values)) > 1e-6: # Only plot if not a zero
        function
        plt.plot(x_values, y_values, label=f'Orthogonal u_{i+1}',
            color=colors[i], linestyle='--', linewidth=2)
    else:
```

```
        print(f"  Orthogonal function u_{i+1} is approximately zero
           and will not be plotted.")

# 4. Evaluate and plot orthonormal functions
print("Plotting orthonormal functions...")
for i, func in enumerate(orthonormal_functions):
    y_values = [func(x) for x in x_values]
    # Check if the orthonormal function is essentially zero
    if np.max(np.abs(y_values)) > 1e-6: # Only plot if not a zero
       function
        plt.plot(x_values, y_values, label=f'Orthonormal e_{i+1}',
           color=colors[i], linestyle=':', linewidth=2)
    else:
        print(f"  Orthonormal function e_{i+1} is approximately
           zero and will not be plotted.")

# 5. Add plot enhancements
plt.title('Functional Gram-Schmidt Orthonormalization')
plt.xlabel('x')
plt.ylabel('Function Value')
plt.grid(True)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left') # Place
   legend outside the plot area
plt.tight_layout()
plt.axhline(0, color='gray', linewidth=0.5)
plt.axvline(0, color='gray', linewidth=0.5)
plt.show()

print("Plotting complete.")
```

Problem: Apply Gram–Schmidt to the basis

$$B = \{1, x, x^2\}$$

in P_2 using the inner product

$$\langle p, q \rangle = \int_{-1}^{1} p(x)q(x)\,dx$$

OUTPUT

Necessary libraries imported successfully.
Functional inner product and norm helper functions defined.

— Input for Functional Gram-Schmidt —
Enter the start point of the integration interval (a): -1
Enter the end point of the integration interval (b): 1
Integration interval set to: [-1.0, 1.0]

Enter the number of functions you want to orthonormalize: 3
You will enter 3 functions.
Enter function f_1 as a string (e.g., 'x', 'x**2', 'np.sin(x)'): 1
Function f_1 ('1') successfully added.
Enter function f_2 as a string (e.g., 'x', 'x**2', 'np.sin(x)'): x
Function f_2 ('x') successfully added.
Enter function f_3 as a string (e.g., 'x', 'x**2', 'np.sin(x)'): x**2
Function f_3 ('x**2') successfully added.

All functions have been entered and validated.

— Functional Gram-Schmidt Orthonormalization Process —

Processing input function f_1 (original: '1')
Orthogonal function u_1 created.
Orthonormal function e_1 created (norm: 1.4142).

Processing input function f_2 (original: 'x')
Subtracted projection onto u_1 (factor: 0.0000)
Orthogonal function u_2 created.
Orthonormal function e_2 created (norm: 0.8165).

Processing input function f_3 (original: 'x**2')
Subtracted projection onto u_1 (factor: 0.3333)
Subtracted projection onto u_2 (factor: 0.0000)
Orthogonal function u_3 created.
Orthonormal function e_3 created (norm: 0.4216).

Functional Gram-Schmidt process completed.

— Symbolic Equivalents (for reference) —
For the input functions '1', 'x', 'x**2' over [-1, 1], the orthonormal functions correspond to scaled Legendre polynomials:
e_1(x) = 1/ (2)
e_2(x) = $\sqrt{\frac{3}{2}}$ * x
e_3(x) = $\sqrt{\frac{5}{8}}$ * (3x^2 - 1)

— Verification of Orthogonality and Normality —
Checking orthogonality of orthonormal functions (inner product should be close to 0):
¡e_1, e_2¿ = 0.00000000
¡e_1, e_3¿ = 0.00000000
¡e_2, e_3¿ = 0.00000000

Checking normality of orthonormal functions (norm should be close to 1):

——e_1—— = 1.00000000
——e_2—— = 1.00000000
——e_3—— = 1.00000000
— Plotting Functional Gram-Schmidt Results —
Plotting original functions...
Plotting orthogonal functions...
Plotting orthonormal functions...

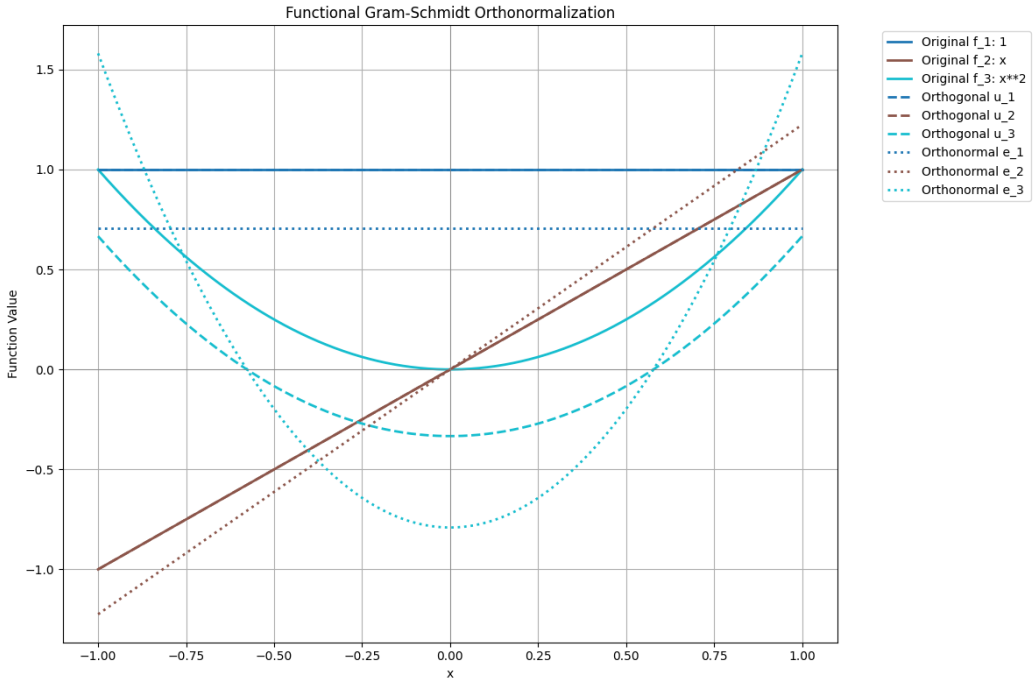

Figure 2: Orthonormalisation of functions

2 Orthogonal Subspaces and Fundamental Subspaces

2.1 Orthogonal Subspaces

Let V be an inner product space over $\mathbb{R}$ (or $\mathbb{C}$). Two subspaces $U, W \subseteq V$ are said to be **orthogonal subspaces** if

$$\langle u, w \rangle = 0 \quad \text{for all } u \in U \text{ and } w \in W.$$

If U and W are orthogonal subspaces of V, then

$$U \cap W = \{0\}.$$

If $x \in U \cap W$, then $x \in U$ and $x \in W$. Hence $\langle x, x \rangle = 0$. Since the inner product is positive definite, $x = 0$.

2.2 Orthogonal Complement

Let W be a subspace of an inner product space V. The **orthogonal complement** of W is defined as

$$W^\perp = \{v \in V \mid \langle v, w\rangle = 0 \text{ for all } w \in W\}.$$

$W^\perp$ is a subspace of V.

[Dimension Formula] If V is finite dimensional, then

$$\dim W + \dim W^\perp = \dim V.$$

In a finite-dimensional inner product space,

$$(W^\perp)^\perp = W.$$

2.3 Orthogonal Decomposition and Direct Sum

Let V be a vector space and U, W subspaces of V. We say that V is the **direct sum** of U and W, written

$$V = U \oplus W,$$

if:

1. $V = U + W$,
2. $U \cap W = \{0\}$.

[Orthogonal Decomposition] If W is a subspace of a finite-dimensional inner product space V, then

$$V = W \oplus W^\perp.$$

For every $v \in V$, there exist unique vectors $w \in W$ and $w^\perp \in W^\perp$ such that

$$v = w + w^\perp.$$

2.4 Fundamental Subspaces of a Matrix

Let A be an $m \times n$ matrix over $\mathbb{R}$.

2.4.1 Column Space

The **column space** of A is

$$\mathrm{Col}(A) = \{Ax \mid x \in \mathbb{R}^n\} \subseteq \mathbb{R}^m.$$

It is the span of the columns of A.

$$\mathrm{rank}(A) = \dim \mathrm{Col}(A).$$

2.4.2 Null Space

The **null space** (kernel) of A is

$$N(A) = \{x \in \mathbb{R}^n \mid Ax = 0\}.$$

$$\text{nullity}(A) = \dim N(A).$$

[Rank–Nullity Theorem]

$$\text{rank}(A) + \text{nullity}(A) = n.$$

2.4.3 Row Space

The **row space** of A is

$$\text{Row}(A) = \text{Col}(A^T) \subseteq \mathbb{R}^n.$$

It is the span of the rows of A.

$$\dim \text{Row}(A) = \text{rank}(A).$$

2.4.4 Left Null Space

The **left null space** of A is

$$N(A^T) = \{y \in \mathbb{R}^m \mid A^T y = 0\}.$$

$$\dim N(A^T) = m - \text{rank}(A).$$

2.5 Orthogonality Relations Among Fundamental Subspaces

For an $m \times n$ matrix A:

1. $N(A)$ is the orthogonal complement of the row space:

$$N(A) = \text{Row}(A)^\perp \subseteq \mathbb{R}^n.$$

2. $N(A^T)$ is the orthogonal complement of the column space:

$$N(A^T) = \text{Col}(A)^\perp \subseteq \mathbb{R}^m.$$

[Fundamental Decompositions]

$$\mathbb{R}^n = \text{Row}(A) \oplus N(A),$$

$$\mathbb{R}^m = \text{Col}(A) \oplus N(A^T).$$

2.6 Dimension Summary

For an $m \times n$ matrix A with rank r:

$$\dim \mathrm{Col}(A) = r,$$
$$\dim \mathrm{Row}(A) = r,$$
$$\dim N(A) = n - r,$$
$$\dim N(A^T) = m - r.$$

These four subspaces are called the **four fundamental subspaces of a matrix**.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
ambient_dimension = int(input("Enter the ambient dimension of the
   vector space: "))

subspace1_vectors = []
num_vectors_subspace1 = int(input(f"Enter the number of spanning
   vectors for Subspace 1 (in R^{ambient_dimension}): "))
for i in range(num_vectors_subspace1):
    vector_str = input(f"Enter elements for vector {i+1} of
       Subspace 1 (space-separated numbers): ")
    vector = [float(x) for x in vector_str.split()]
    if len(vector) != ambient_dimension:
        print(f"Warning: Vector {i+1} has {len(vector)} dimensions,
           but ambient dimension is {ambient_dimension}. Please
           re-enter.")
        i -= 1 # Decrement to re-prompt for the same vector
        continue
    subspace1_vectors.append(vector)

subspace2_vectors = []
num_vectors_subspace2 = int(input(f"Enter the number of spanning
   vectors for Subspace 2 (in R^{ambient_dimension}): "))
for i in range(num_vectors_subspace2):
    vector_str = input(f"Enter elements for vector {i+1} of
       Subspace 2 (space-separated numbers): ")
    vector = [float(x) for x in vector_str.split()]
    if len(vector) != ambient_dimension:
        print(f"Warning: Vector {i+1} has {len(vector)} dimensions,
           but ambient dimension is {ambient_dimension}. Please
           re-enter.")
        i -= 1 # Decrement to re-prompt for the same vector
        continue
    subspace2_vectors.append(vector)
```

```
print(f"Ambient Dimension: {ambient_dimension}")
print(f"Subspace 1 Vectors: {subspace1_vectors}")
print(f"Subspace 2 Vectors: {subspace2_vectors}")
# Convert the collected spanning vectors into NumPy arrays
matrix_subspace1 = np.array(subspace1_vectors)
matrix_subspace2 = np.array(subspace2_vectors)

print(f"Matrix Subspace 1:\n{matrix_subspace1}")
print(f"Matrix Subspace 2:\n{matrix_subspace2}")

# Calculate the dot product of each vector in matrix_subspace1 with
   each vector in matrix_subspace2
dot_product_matrix = np.dot(matrix_subspace1, matrix_subspace2.T)

print(f"Dot Product Matrix:\n{dot_product_matrix}")

# Check if all elements in the resulting dot product matrix are
   close to zero
are_orthogonal = np.allclose(dot_product_matrix, 0, atol=1e-9)

# Print the result
print(f"Are the two subspaces orthogonal? {are_orthogonal}")



# Create a new figure and an Axes3D subplot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# --- Plotting Subspace 1 (Plane) ---
# Spanning vectors for Subspace 1
v1 = matrix_subspace1[0]
v2 = matrix_subspace1[1]

# Plot spanning vectors of Subspace 1 as arrows
ax.quiver(0, 0, 0, v1[0], v1[1], v1[2], color='red',
   arrow_length_ratio=0.1, label='Subspace 1 Vector 1')
ax.quiver(0, 0, 0, v2[0], v2[1], v2[2], color='darkred',
   arrow_length_ratio=0.1, label='Subspace 1 Vector 2')

# Create a grid of points for the plane spanned by v1 and v2
s_vals = np.linspace(-1, 1, 10) # Scalar multipliers for v1
t_vals = np.linspace(-1, 1, 10) # Scalar multipliers for v2
S, T = np.meshgrid(s_vals, t_vals)

X_plane = S * v1[0] + T * v2[0]
Y_plane = S * v1[1] + T * v2[1]
```

```
Z_plane = S * v1[2] + T * v2[2]

# Plot the plane spanned by Subspace 1
ax.plot_surface(X_plane, Y_plane, Z_plane, alpha=0.5, color='red',
    rstride=1, cstride=1, label='Subspace 1 (Plane)')

# --- Plotting Subspace 2 (Line) ---
# Spanning vector for Subspace 2
v3 = matrix_subspace2[0]

# Plot spanning vector of Subspace 2 as an arrow
ax.quiver(0, 0, 0, v3[0], v3[1], v3[2], color='blue',
    arrow_length_ratio=0.1, label='Subspace 2 Vector 1')

# Generate points along the line spanned by v3
t_line = np.linspace(-2, 2, 100) # Scalar multipliers for v3
X_line = t_line * v3[0]
Y_line = t_line * v3[1]
Z_line = t_line * v3[2]

# Plot the line spanned by Subspace 2
ax.plot(X_line, Y_line, Z_line, color='blue', linewidth=2,
    label='Subspace 2 (Line)')

# --- Plotting Configuration ---
# Set labels
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_zlabel('Z-axis')

# Set title based on orthogonality result
ax.set_title(f'Subspace Visualization (Orthogonal:
    {are_orthogonal})')

# Add a legend
# Due to a known issue with plot_surface and legends in older
    matplotlib versions,
# a workaround is often needed to include the surface in the legend.
# For simplicity, we manually create a proxy artist for the surface
    legend.
from matplotlib.patches import Patch
red_patch = Patch(color='red', alpha=0.5, label='Subspace 1
    (Plane)')
blue_line = plt.Line2D([0],[0], linestyle='-', color='blue',
    linewidth=2, label='Subspace 2 (Line)')

# Combine handles for arrows and the manually created patches/lines
```

```
    for the subspaces
handles, labels = ax.get_legend_handles_labels()
# Remove default legend entries for plot_surface and plot if they
    were added incorrectly
handles = [h for h, l in zip(handles, labels) if not (l ==
    'Subspace 1 (Plane)' or l == 'Subspace 2 (Line)')]
labels = [l for l in labels if not (l == 'Subspace 1 (Plane)' or l
    == 'Subspace 2 (Line)')]

ax.legend(handles + [red_patch, blue_line], labels +
    [red_patch.get_label(), blue_line.get_label()], loc='best')

# Set equal aspect ratio for better visualization (optional)
# ax.set_box_aspect([1,1,1]) # requires matplotlib >= 3.3.0

plt.grid(True)
plt.show()
```

Problem: Determine whether the subspaces

$$S_1 = \text{span}\left\{\begin{bmatrix}2\\1\\-1\end{bmatrix}, \begin{bmatrix}0\\1\\1\end{bmatrix}\right\}, \qquad S_2 = \text{span}\left\{\begin{bmatrix}-1\\2\\0\end{bmatrix}\right\}$$

are orthogonal.

OUTPUT

Enter the ambient dimension of the vector space: 3
Enter the number of spanning vectors for Subspace 1 (in R$\hat{3}$): 2
Enter elements for vector 1 of Subspace 1 (space-separated numbers): 2 1 -1
Enter elements for vector 2 of Subspace 1 (space-separated numbers): 0 1 1
Enter the number of spanning vectors for Subspace 2 (in R$\hat{3}$): 1
Enter elements for vector 1 of Subspace 2 (space-separated numbers): -1 2 0
Ambient Dimension: 3
Subspace 1 Vectors: [[2.0, 1.0, -1.0], [0.0, 1.0, 1.0]]
Subspace 2 Vectors: [[-1.0, 2.0, 0.0]]
Matrix Subspace 1:
[[2. 1. -1.]
[0. 1. 1.]]
Matrix Subspace 2:
[[-1. 2. 0.]]
Dot Product Matrix:
[[0.]
[2.]]
Are the two subspaces orthogonal? False

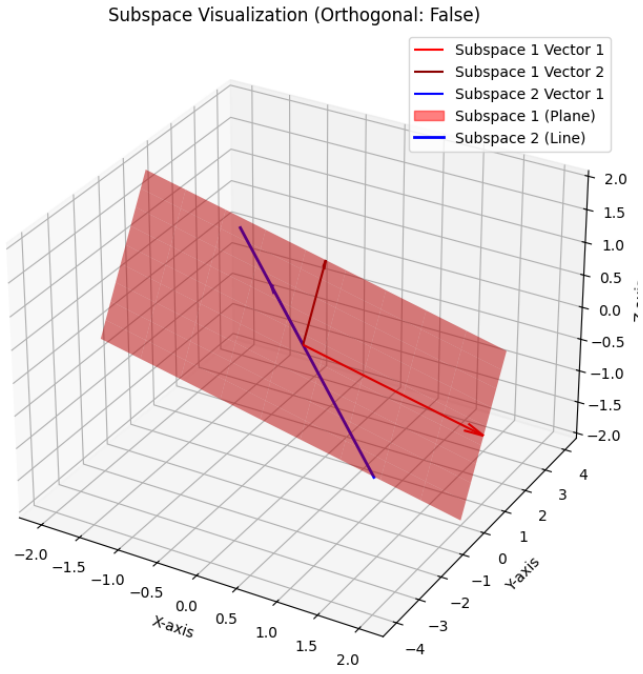

Figure 3:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import scipy.linalg as la

n = int(input("Enter the dimension (n) of the vector space: "))
print(f"Vector space dimension (n): {n}")
while True:
    try:
        k = int(input(f"Enter the number of vectors (k) for the
            subspace (k <= {n}): "))
        if 1 <= k <= n:
            break
        else:
            print(f"Please enter a value for k such that 1 <= k <=
                {n}.")
    except ValueError:
        print("Invalid input. Please enter an integer.")

print(f"Number of vectors for the subspace (k): {k}")

subspace_vectors = []
print(f"\nNow, enter the {n} numerical elements for each of the {k}
    vectors.")
```

```
for i in range(k):
    while True:
        try:
            vector_str = input(f"Enter {n} numerical elements for
               vector {i+1} (space or comma separated): ")
            # Replace commas with spaces to handle both delimiters
            vector_elements = [float(x) for x in
               vector_str.replace(',', ' ').split()]

            if len(vector_elements) == n:
                subspace_vectors.append(np.array(vector_elements))
                break
            else:
                print(f"Please enter exactly {n} numerical
                   elements.")
        except ValueError:
            print("Invalid input. Please ensure all elements are
               numerical.")

print("\nSubspace vectors entered:")
for i, vec in enumerate(subspace_vectors):
    print(f"Vector {i+1}: {vec}")

# Convert the list of subspace vectors into a NumPy array
# Each vector will be a row in the matrix
A = np.array(subspace_vectors)

# Compute the null space of the matrix A
# The columns of null_space_basis form an orthonormal basis for the
   null space of A
null_space_basis = la.null_space(A)

print("Matrix A (formed from subspace vectors):")
print(A)
print("\nBasis for the Orthogonal Complement (Null Space of A):")
if null_space_basis.size == 0:
    print("The orthogonal complement is the zero vector space (only
       contains the zero vector).")
else:
    for i in range(null_space_basis.shape[1]):
        print(f"Basis vector {i+1}: {null_space_basis[:, i]}")

if n > 3:
    print(f"The vector space dimension is {n}, which is greater
       than 3. Direct visualization is not possible.\n")
    print("Properties of the Orthogonal Complement:")
```

```
    print("1. The orthogonal complement (W_perp) of a subspace W
       consists of all vectors that are orthogonal to every vector
       in W.")
    print("2. The intersection of W and W_perp is the zero vector
       space= {0}.")
    print("3. The sum of the dimension of W and the dimension of
       W_perp equals the dimension of the entire vector space:
       dim(W) + dim(W_perp) = n.")
    print("4. Every vector in the entire vector space can be
       uniquely expressed as the sum of a vector from W and a
       vector from W_perp.")
else:
    fig = plt.figure(figsize=(10, 8))
    if n == 2:
        ax = fig.add_subplot(111)
    else: # n == 3
        ax = fig.add_subplot(111, projection='3d')

    # Set axis limits
    limit = 2.0 # Adjust as needed for better visualization
    if n == 2:
        ax.set_xlim([-limit, limit])
        ax.set_ylim([-limit, limit])
    else: # n == 3
        ax.set_xlim([-limit, limit])
        ax.set_ylim([-limit, limit])
        ax.set_zlim([-limit, limit])

    # Plot the XZ-plane (subspace) if n == 3
    if n == 3:
        x_plane = np.linspace(-limit, limit, 10)
        z_plane = np.linspace(-limit, limit, 10)
        X_plane, Z_plane = np.meshgrid(x_plane, z_plane)
        Y_plane = np.zeros_like(X_plane) # Y is 0 for XZ-plane
        ax.plot_surface(X_plane, Y_plane, Z_plane, alpha=0.2,
           color='lightblue', linewidth=0)
        # Dummy plot for legend entry as plot_surface doesn't
           automatically add to legend
        ax.plot([], [], [], color='lightblue', alpha=0.2,
           linewidth=10, label='Subspace (XZ-plane)')

        # Explicitly plot the Y-axis as the orthogonal complement
           line for clarity
        ax.plot([0, 0], [-limit, limit], [0, 0], color='purple',
           linestyle='--', linewidth=2, label='Orthogonal
           Complement (Y-Axis Line)')
```

```
    # Plot original subspace vectors
    for i, vec in enumerate(subspace_vectors):
        if n == 2:
            ax.quiver(0, 0, vec[0], vec[1], color='b',
               arrow_length_ratio=0.1, label=f'Subspace Vector
               {i+1}')
        else: # n == 3
            ax.quiver(0, 0, 0, vec[0], vec[1], vec[2], color='b',
               arrow_length_ratio=0.1, label=f'Subspace Vector
               {i+1}')

    # Plot orthogonal complement basis vectors
    if null_space_basis.size > 0:
        for i in range(null_space_basis.shape[1]):
            basis_vec = null_space_basis[:, i]
            if n == 2:
                ax.quiver(0, 0, basis_vec[0], basis_vec[1],
                   color='r', arrow_length_ratio=0.1,
                   label=f'Orthogonal Complement Basis {i+1}')
            else: # n == 3
                ax.quiver(0, 0, 0, basis_vec[0], basis_vec[1],
                   basis_vec[2], color='r', arrow_length_ratio=0.1,
                   label=f'Orthogonal Complement Basis {i+1}')

    # Add labels and title
    ax.set_xlabel('X-axis')
    ax.set_ylabel('Y-axis')
    if n == 3:
        ax.set_zlabel('Z-axis')
    ax.set_title('Subspace and its Orthogonal Complement')
    ax.legend()
    ax.grid(True)
    plt.show()
```

Probelm: Let S be the subspace of $\mathbb{R}^3$ consisting of the xz-plane.

Find the orthogonal complement $S^{\perp}$.

OUTPUT

Enter the dimension (n) of the vector space: 3
Vector space dimension (n): 3
Enter the number of vectors (k) for the subspace (k ¡= 3): 2
Number of vectors for the subspace (k): 2

Now, enter the 3 numerical elements for each of the 2 vectors.
Enter 3 numerical elements for vector 1 (space or comma separated): 1 0 0

Enter 3 numerical elements for vector 2 (space or comma separated): 0 0 1

Subspace vectors entered:
Vector 1: [1. 0. 0.]
Vector 2: [0. 0. 1.]
Matrix A (formed from subspace vectors):
[[1. 0. 0.]
0. 0. 1.]]
Basis for the Orthogonal Complement (Null Space of A):
Basis vector 1: [0. -1. 0.]

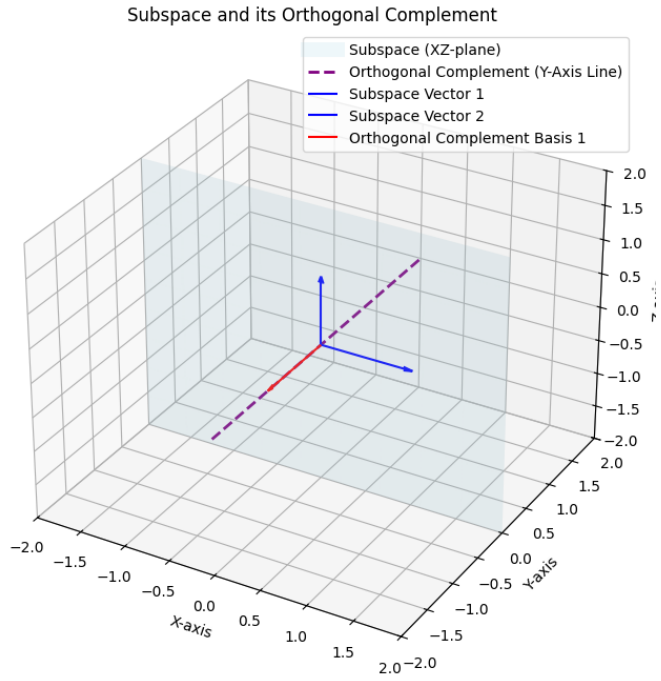

Figure 4:

3 Orthogonal Projection of a Vector onto a Subspace

3.1 Projection onto a Vector

Let V be an inner product space and let $u \in V$ with $u \neq 0$.

The **orthogonal projection** of a vector $v \in V$ onto u is defined as

$$\text{proj}_u(v) = \frac{\langle v, u \rangle}{\langle u, u \rangle} u.$$

The vector v can be decomposed uniquely as

$$v = \text{proj}_u(v) + e,$$

where e is orthogonal to u, i.e.,

$$\langle e, u \rangle = 0.$$

3.2 Projection onto a Subspace

Let W be a subspace of a finite-dimensional inner product space V.

[Orthogonal Decomposition] For every $v \in V$, there exist unique vectors

$$w \in W \quad \text{and} \quad z \in W^{\perp}$$

such that

$$v = w + z.$$

The vector w in the above decomposition is called the **orthogonal projection of** v **onto** W, and is denoted by

$$\text{proj}_W(v).$$

Thus,

$$v - \text{proj}_W(v) \in W^{\perp}.$$

3.3 Projection Using an Orthonormal Basis

Let $\{w_1, w_2, \ldots, w_k\}$ be an orthonormal basis for W.

For any $v \in V$,

$$\text{proj}_W(v) = \sum_{i=1}^{k} \langle v, w_i \rangle w_i.$$

Since $\{w_1, \ldots, w_k\}$ is an orthonormal basis, any $w \in W$ can be written as

$$w = \sum_{i=1}^{k} c_i w_i.$$

Taking inner product with w_j and using orthonormality,

$$\langle v - w, w_j \rangle = 0$$

gives

$$c_j = \langle v, w_j \rangle.$$

Substituting yields the formula.

3.4 Matrix Formula for Projection onto Column Space

Let A be an $m \times n$ matrix with linearly independent columns. Then the orthogonal projection of $b \in \mathbb{R}^m$ onto $\text{Col}(A)$ is

$$\hat{b} = A(A^T A)^{-1} A^T b.$$

The matrix

$$P = A(A^T A)^{-1} A^T$$

is called the **projection matrix** onto $\text{Col}(A)$.

3.5 Properties of the Projection Matrix

1. $P^2 = P$ (idempotent property)
2. $P^T = P$ (symmetric)
3. Range(P) = Col(A)
4. Null$(P) = N(A^T)$

3.6 Least Squares Interpretation

For $b \in \mathbb{R}^m$, the projection $\hat{b}$ is the unique vector in Col(A) that minimizes

$$\|b - Ax\|.$$

The minimizing vector satisfies the **normal equations**:

$$A^T Ax = A^T b.$$

Thus,

$$\hat{b} = A\hat{x}.$$

3.7 Fundamental Orthogonal Decomposition

For an $m \times n$ matrix A,

$$\mathbb{R}^m = \text{Col}(A) \oplus N(A^T).$$

Hence every $b \in \mathbb{R}^m$ can be written uniquely as

$$b = \hat{b} + e,$$

where

$$\hat{b} \in \text{Col}(A), \quad e \in N(A^T).$$

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 1. Ask for the dimension of the vector space
while True:
    try:
        n = int(input("Enter the dimension of the vector space
           (e.g., 3 for R^3): "))
        if n <= 0:
            print("Dimension must be a positive integer.")
        else:
```

```
            break
    except ValueError:
        print("Invalid input. Please enter an integer.")

print(f"Vector space dimension: {n}")

# 2. Ask for the number of vectors that form the span of the
   subspace
while True:
    try:
        num_vectors = int(input(f"Enter the number of vectors that
           span the subspace (must be <= {n}): "))
        if num_vectors <= 0 or num_vectors > n:
            print(f"Number of vectors must be a positive integer
               and not exceed the dimension {n}.")
        else:
            break
    except ValueError:
        print("Invalid input. Please enter an integer.")

print(f"Number of spanning vectors: {num_vectors}")

# 3. Ask to enter the vector elements for each spanning vector
spanning_vectors = []
for i in range(num_vectors):
    while True:
        try:
            elements_str = input(f"Enter the {n} elements for
               spanning vector {i+1}, separated by spaces: ")
            elements = list(map(float, elements_str.split()))
            if len(elements) != n:
                print(f"Please enter exactly {n} elements.")
            else:
                spanning_vectors.append(np.array(elements))
                break
        except ValueError:
            print("Invalid input. Please enter numbers separated by
               spaces.")

A = np.vstack(spanning_vectors).T # Each column of A is a spanning
   vector
print("\nSpanning vectors (as columns of matrix A):\n", A)

# 4. Ask to enter the vector which you want to project
while True:
    try:
        elements_str = input(f"\nEnter the {n} elements for the
```

```
            vector to project (v), separated by spaces: ")
        v_elements = list(map(float, elements_str.split()))
        if len(v_elements) != n:
                print(f"Please enter exactly {n} elements.")
        else:
            v = np.array(v_elements)
            break
    except ValueError:
        print("Invalid input. Please enter numbers separated by
            spaces.")

print("Vector to project (v):", v)

# 5. Calculate the projection
# The projection of vector v onto the subspace spanned by columns
    of A is given by:
# P_A(v) = A * (A^T * A)^-1 * A^T * v

# Ensure the spanning vectors are linearly independent, otherwise
    A^T * A might be singular.
# If they are not linearly independent, `np.linalg.inv` will raise
    an error.
# For a robust solution, one would use QR decomposition or SVD.

try:
    # Calculate A_transpose * A
    ATA = A.T @ A

    # Calculate (A_transpose * A)^-1
    ATA_inv = np.linalg.inv(ATA)

    # Calculate the projection matrix P = A * (A^T * A)^-1 * A.T
    projection_matrix = A @ ATA_inv @ A.T

    # Calculate the projected vector
    projection_of_v = projection_matrix @ v

    print("\nProjection matrix (P):\n", projection_matrix)
    print("\nThe projection of vector v onto the subspace is:",
        projection_of_v)

except np.linalg.LinAlgError as e:
    print(f"\nError: {e}")
    print("This might happen if the spanning vectors are not
        linearly independent.")
    print("Cannot compute the inverse of (A^T * A).")
    # Set projection_of_v to None to avoid plotting errors if
```

```
        calculation failed
    projection_of_v = None
except Exception as e:
    print(f"An unexpected error occurred: {e}")
    projection_of_v = None


# --- START PLOTTING CODE ---
if projection_of_v is not None: # Only proceed with plotting if
   projection was successful
    if n == 3: # Plot in 3D
        fig = plt.figure(figsize=(10, 8))
        ax = fig.add_subplot(111, projection='3d')

        def plot_vector_3d(ax, vector, color, label, linestyle='-',
           alpha=1.0):
            ax.quiver(0, 0, 0, vector[0], vector[1], vector[2],
                      color=color, arrow_length_ratio=0.1,
                         label=label, linestyle=linestyle,
                         alpha=alpha)

        # 1. Plot spanning vectors
        for i, vec in enumerate(spanning_vectors):
            plot_vector_3d(ax, vec, color=f'C{i}', label=f'Spanning
               Vector {i+1}')

        # 1.5 Plot the span (plane or line)
        if num_vectors == 2 and
           np.linalg.matrix_rank(np.array(spanning_vectors).T) == 2:
            # Span is a plane (assuming linearly independent
               vectors)
            s1 = spanning_vectors[0]
            s2 = spanning_vectors[1]

            # Generate points for the plane using linear
               combinations of s1 and s2
            u_vals = np.linspace(-1, 1, 10)
            w_vals = np.linspace(-1, 1, 10)
            U, W = np.meshgrid(u_vals, w_vals)

            X = U * s1[0] + W * s2[0]
            Y = U * s1[1] + W * s2[1]
            Z = U * s1[2] + W * s2[2]

            ax.plot_surface(X, Y, Z, alpha=0.3, color='gray',
               rstride=1, cstride=1)
            # Add a dummy point for the legend entry of the plane
```

```
        ax.scatter([], [], [], color='gray', alpha=0.3,
            label='Subspace Span (Plane)')

    elif num_vectors == 1:
        # Span is a line
        s1 = spanning_vectors[0]
        line_points = np.outer(np.linspace(-2, 2, 100), s1) #
            Extend the line
        ax.plot(line_points[:, 0], line_points[:, 1],
            line_points[:, 2],
                color='gray', linestyle='--', label='Subspace
                    Span (Line)')
    elif num_vectors > 2:
        print("For 3D space, plotting the full span of more
            than 2 linearly independent vectors as a simple
            surface is complex.")
        print("Only the individual spanning vectors are
            plotted.")

    # 2. Plot the vector to project
    plot_vector_3d(ax, v, color='r', label='Vector to Project
        (v)')

    # 3. Plot the projection vector
    plot_vector_3d(ax, projection_of_v, color='g',
        label='Projection of v', linestyle='--')

    # Add a dashed line from v to its projection for visual
        clarity (error vector)
    error_vector_start = v
    error_vector_end = projection_of_v
    ax.plot([error_vector_start[0], error_vector_end[0]],
            [error_vector_start[1], error_vector_end[1]],
            [error_vector_start[2], error_vector_end[2]],
            color='purple', linestyle=':', label='Error Vector
                (v - proj_v)')

    # Set labels and title
    ax.set_xlabel('X-axis')
    ax.set_ylabel('Y-axis')
    ax.set_zlabel('Z-axis')
    ax.set_title('Projection of a Vector onto a Subspace')
    ax.legend()
    ax.grid(True)
    ax.set_aspect('auto') # or 'equal' if desired

    # Set limits to ensure all vectors are visible, centered
```

```
            around origin
        all_vectors = np.array(spanning_vectors + [v,
            projection_of_v])
        max_coord = np.max(np.abs(all_vectors)) if all_vectors.size
            > 0 else 1.0
        ax.set_xlim([-max_coord*1.2, max_coord*1.2])
        ax.set_ylim([-max_coord*1.2, max_coord*1.2])
        ax.set_zlim([-max_coord*1.2, max_coord*1.2])

        plt.show()

    elif n == 2: # Plot in 2D
        fig, ax = plt.subplots(figsize=(8, 6))

        def plot_vector_2d(ax, vector, color, label, linestyle='-',
            alpha=1.0):
            ax.quiver(0, 0, vector[0], vector[1],
                      angles='xy', scale_units='xy', scale=1,
                      color=color, label=label,
                         linestyle=linestyle, alpha=alpha)

        # 1. Plot spanning vectors
        for i, vec in enumerate(spanning_vectors):
            plot_vector_2d(ax, vec, color=f'C{i}', label=f'Spanning
                Vector {i+1}')

        # 1.5 Plot the span (line or full plane indication)
        if num_vectors >= 1 and
            np.linalg.matrix_rank(np.array(spanning_vectors).T) == 1:
            # If spanning a line (one effective basis vector)
            s_effective = spanning_vectors[0] # Use the first one
                as representative
            # Extend the line
            all_coords = np.concatenate([v, projection_of_v,
                *spanning_vectors])
            line_extent = np.max(np.abs(all_coords)) * 2 if
                all_coords.size > 0 else 2.0
            line_x = np.linspace(-line_extent, line_extent, 100)
            if s_effective[0] != 0:
                line_y = (s_effective[1] / s_effective[0]) * line_x
            else: # Vertical line
                line_y = np.linspace(-line_extent, line_extent, 100)
                line_x = np.zeros_like(line_y)
            ax.plot(line_x, line_y, color='gray', linestyle='--',
                label='Subspace Span (Line)')
        elif num_vectors >= 2 and
            np.linalg.matrix_rank(np.array(spanning_vectors).T) == 2:
```

```
            # Span is the entire 2D plane
            ax.text(0.5, 0.95, 'Subspace Span: Entire 2D Plane',
                transform=ax.transAxes,
                     horizontalalignment='center',
                        verticalalignment='top', color='gray')


        # 2. Plot the vector to project
        plot_vector_2d(ax, v, color='r', label='Vector to Project
            (v)')

        # 3. Plot the projection vector
        plot_vector_2d(ax, projection_of_v, color='g',
            label='Projection of v', linestyle='--')

        # Add a dashed line from v to its projection
        ax.plot([v[0], projection_of_v[0]],
                [v[1], projection_of_v[1]],
                color='purple', linestyle=':', label='Error Vector
                    (v - proj_v)')


        # Set labels and title
        ax.set_xlabel('X-axis')
        ax.set_ylabel('Y-axis')
        ax.set_title('Projection of a Vector onto a Subspace (2D)')
        ax.legend()
        ax.grid(True)
        ax.set_aspect('equal', adjustable='box')

        # Set limits
        all_vectors = np.array(spanning_vectors + [v,
            projection_of_v])
        max_coord = np.max(np.abs(all_vectors)) if all_vectors.size
            > 0 else 1.0
        ax.set_xlim([-max_coord*1.2, max_coord*1.2])
        ax.set_ylim([-max_coord*1.2, max_coord*1.2])

        plt.show()
    else:
        print(f"Plotting is currently supported only for 2D (n=2)
            and 3D (n=3) vector spaces. Current dimension: {n}")
else:
    print("Projection could not be calculated, so plotting cannot
        proceed.")
```

Problem: Let

$$S = \text{span}\left\{ \begin{bmatrix} 0 \\ 0 \\ -1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} \right\}, \qquad v = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \end{bmatrix}.$$

Find the projection of the vector v onto the subspace S.

OUTPUT

Enter the dimension of the vector space (e.g., 3 for R$\hat{3}$): 4
Vector space dimension: 4
Enter the number of vectors that span the subspace (must be ¡= 4): 2
Number of spanning vectors: 2
Enter the 4 elements for spanning vector 1, separated by spaces: 0 0 -1 1
Enter the 4 elements for spanning vector 2, separated by spaces: 0 1 1 1

Spanning vectors (as columns of matrix A):
[[0. 0.]
[0. 1.]
[-1. 1.]
[1. 1.]]

Enter the 4 elements for the vector to project (v), separated by spaces: 1 0 1 1
Vector to project (v): [1. 0. 1. 1.]

Projection matrix (P):
[[0. 0. 0. 0.]
[0. 0.33333333 0.33333333 0.33333333]
[0. 0.33333333 0.83333333 -0.16666667]
[0. 0.33333333 -0.16666667 0.83333333]]

The projection of vector v onto the subspace is: [0. 0.66666667 0.66666667 0.66666667]

Plotting is currently supported only for 2D (n=2) and 3D (n=3) vector spaces.Current dimension: 4

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def get_matrix_input():
    while True:
        try:
```

```
            rows = int(input("Enter the number of rows for the
               matrix: "))
            cols = int(input("Enter the number of columns for the
               matrix: "))
            if rows <= 0 or cols <= 0:
                raise ValueError
            break
        except ValueError:
            print("Invalid input. Please enter positive integers
               for rows and columns.")

    matrix = []
    print(f"Enter the elements of the {rows}x{cols} matrix row by
       row:")
    for i in range(rows):
        row = []
        for j in range(cols):
            while True:
                try:
                    element = float(input(f"Enter element
                       M[{i+1}][{j+1}]: "))
                    row.append(element)
                    break
                except ValueError:
                    print("Invalid input. Please enter a number.")
        matrix.append(row)
    return np.array(matrix)

def get_vector_input(dimension):
    vector = []
    print(f"Enter the elements of the vector (dimension
       {dimension}):")
    for i in range(dimension):
        while True:
            try:
                element = float(input(f"Enter element V[{i+1}]: "))
                vector.append(element)
                break
            except ValueError:
                print("Invalid input. Please enter a number.")
    return np.array(vector)

# Get matrix A from user
A = get_matrix_input()
print("Your matrix A:")
print(A)
```

```
# Get vector b from user
b = get_vector_input(A.shape[0])
print("Your vector b:")
print(b)

# Calculate the projection
def project_vector_onto_column_space(A, b):
    try:
        # The projection matrix P = A @ (A.T @ A)^-1 @ A.T
        # The projected vector p = P @ b
        # This can be more numerically stable by solving A.T @ A @
           x = A.T @ b for x, then p = A @ x

        # Check if A.T @ A is invertible (i.e., A has linearly
           independent columns)
        ATA = A.T @ A
        if np.linalg.det(ATA) == 0:
            print("Warning: Columns of matrix A are not linearly
               independent. Projection might not be unique or
               well-defined by this method.")
            print("Attempting generalized inverse...")
            # Use pseudo-inverse for A.T @ A if it's singular
            x = np.linalg.pinv(ATA) @ A.T @ b
        else:
            x = np.linalg.solve(ATA, A.T @ b)

        projection = A @ x
        return projection
    except np.linalg.LinAlgError as e:
        print(f"An error occurred during projection calculation:
           {e}")
        return None

projected_vector = project_vector_onto_column_space(A, b)

if projected_vector is not None:
    print("\nOriginal vector b:", b)
    print("Projected vector p:", projected_vector)

# Plotting the vectors and column space

def plot_projection(A, b, p):
    dimension = A.shape[0]

    if dimension == 2:
        fig, ax = plt.subplots(figsize=(8, 8))
        ax.set_aspect('equal', adjustable='box')
```

```
ax.set_title('Vector Projection in 2D')
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.grid(True)

# Plot origin
ax.plot(0, 0, 'ko')

# Plot original vector b
ax.quiver(0, 0, b[0], b[1], angles='xy', scale_units='xy',
   scale=1, color='b', label='Original Vector b')

# Plot projected vector p
ax.quiver(0, 0, p[0], p[1], angles='xy', scale_units='xy',
   scale=1, color='r', label='Projected Vector p')

# Plot the column space (line(s))
# For 2D, the column space is a line if rank is 1, or the
   whole R2 if rank is 2.
rank = np.linalg.matrix_rank(A)
if rank == 1:
    # Get the basis vector for the column space (first
       column if it's non-zero)
    basis_vector = A[:, 0]
    if np.linalg.norm(basis_vector) == 0: # If the first
       column is zero, try another
        for col_idx in range(A.shape[1]):
            if np.linalg.norm(A[:, col_idx]) != 0:
                basis_vector = A[:, col_idx]
                break
    if np.linalg.norm(basis_vector) != 0:
        # Plot line through origin in direction of
           basis_vector
        # Extend the line to cover the plot area
        x_line = np.linspace(-10, 10, 100) # Adjust range
           as needed
        y_line = (basis_vector[1] / basis_vector[0]) *
           x_line if basis_vector[0] != 0 else
           np.sign(basis_vector[1]) * x_line
        ax.plot(x_line, y_line, 'g--', label='Column Space
           of A')
    else:
        print("Column space is just the origin.")
elif rank == 2:
    # If rank is 2 in 2D, the column space is the entire
       plane, which is implicitly shown by the axes
    print("Column space is the entire 2D plane.")
```

```
        # Plot the error vector (b - p)
        ax.quiver(p[0], p[1], b[0] - p[0], b[1] - p[1],
            angles='xy', scale_units='xy', scale=1, color='orange',
            linestyle=':', label='Error Vector (b-p)')

        # Set plot limits dynamically
        all_points = np.vstack(([0,0], b, p))
        max_val = np.max(np.abs(all_points)) * 1.2
        ax.set_xlim(-max_val, max_val)
        ax.set_ylim(-max_val, max_val)

        ax.legend()
        plt.show()

    elif dimension == 3:
        fig = plt.figure(figsize=(10, 10))
        ax = fig.add_subplot(111, projection='3d')
        ax.set_title('Vector Projection in 3D')
        ax.set_xlabel('X-axis')
        ax.set_ylabel('Y-axis')
        ax.set_zlabel('Z-axis')

        # Plot origin
        ax.scatter(0, 0, 0, color='k', s=50, label='Origin')

        # Plot original vector b
        ax.quiver(0, 0, 0, b[0], b[1], b[2], color='b', length=1,
            normalize=False, arrow_length_ratio=0.1, label='Original
            Vector b')

        # Plot projected vector p
        ax.quiver(0, 0, 0, p[0], p[1], p[2], color='r', length=1,
            normalize=False, arrow_length_ratio=0.1,
            label='Projected Vector p')

        # Plot the column space
        rank = np.linalg.matrix_rank(A)
        if rank == 1:
            # Column space is a line through the origin
            basis_vector = A[:, 0]
            if np.linalg.norm(basis_vector) == 0: # If the first
                column is zero, try another
                for col_idx in range(A.shape[1]):
                    if np.linalg.norm(A[:, col_idx]) != 0:
                        basis_vector = A[:, col_idx]
                        break
```

```
        if np.linalg.norm(basis_vector) != 0:
            t = np.linspace(-2, 2, 100) # Parameter for the line
            line_x = basis_vector[0] * t
            line_y = basis_vector[1] * t
            line_z = basis_vector[2] * t
            ax.plot(line_x, line_y, line_z, 'g--',
               label='Column Space of A (Line)')
        else:
            print("Column space is just the origin.")

    elif rank == 2:
        # Column space is a plane through the origin
        # Find two linearly independent vectors spanning the
           column space
        # These are usually two columns of A if A has rank 2
        # Or use SVD to get basis vectors for column space
        U, s, Vh = np.linalg.svd(A)
        basis1 = U[:, 0] # First singular vector of U (left
           singular vectors are basis for column space)
        basis2 = U[:, 1] # Second singular vector

        # Set plot limits dynamically to determine plane extent
        all_points = np.vstack(([0,0,0], b, p))
        max_val = np.max(np.abs(all_points)) * 1.2 # Use the
           same max_val as for axes limits

        # Create a meshgrid for the plane, scaled by max_val
        s = np.linspace(-max_val, max_val, 10)
        t = np.linspace(-max_val, max_val, 10)
        S, T = np.meshgrid(s, t)

        plane_x = S * basis1[0] + T * basis2[0]
        plane_y = S * basis1[1] + T * basis2[1]
        plane_z = S * basis1[2] + T * basis2[2]

        ax.plot_surface(plane_x, plane_y, plane_z, alpha=0.6,
           color='lightgreen', label='Column Space of A
           (Plane)', shade=False)
        # Trick to make label appear for plot_surface
        ax.plot([], [], [], color='lightgreen', alpha=0.6,
           label='Column Space of A (Plane)')

    elif rank == 3:
        print("Column space is the entire 3D space.")

    # Plot the error vector (b - p)
    error_vec = b - p
```

```
        ax.quiver(p[0], p[1], p[2], error_vec[0], error_vec[1],
            error_vec[2], color='orange', length=1, normalize=False,
            linestyle=':', arrow_length_ratio=0.1, label='Error
            Vector (b-p)')

        # Set plot limits dynamically
        all_points = np.vstack(([0,0,0], b, p))
        max_val = np.max(np.abs(all_points)) * 1.2
        ax.set_xlim([-max_val, max_val])
        ax.set_ylim([-max_val, max_val])
        ax.set_zlim([-max_val, max_val])

        # Adjust the camera angle for better visualization
        ax.view_init(elev=20, azim=45)

        ax.legend()
        plt.show()

    else:
        print("Plotting is currently supported only for 2D and 3D
            vectors.")

if projected_vector is not None:
    plot_projection(A, b, projected_vector)
```

Problem: Let

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 2 \\ -2 \\ 1 \end{bmatrix}.$$

Find the orthogonal projection of b onto the column space of A.

OUTPUT

Enter the number of rows for the matrix: 3
Enter the number of columns for the matrix: 2
Enter the elements of the 3x2 matrix row by row:
Enter element M[1][1]: 1
Enter element M[1][2]: 2
Enter element M[2][1]: 0
Enter element M[2][2]: 1
Enter element M[3][1]: 1
Enter element M[3][2]: 1
Your matrix A:
[[1. 2.]
[0. 1.]

[1. 1.]]
Enter the elements of the vector (dimension 3):
Enter element V[1]: 2
Enter element V[2]: -2
Enter element V[3]: 1
Your vector b:
[2. -2. 1.]

Original vector b: [2. -2. 1.]
Projected vector p: [1. -1. 2.]

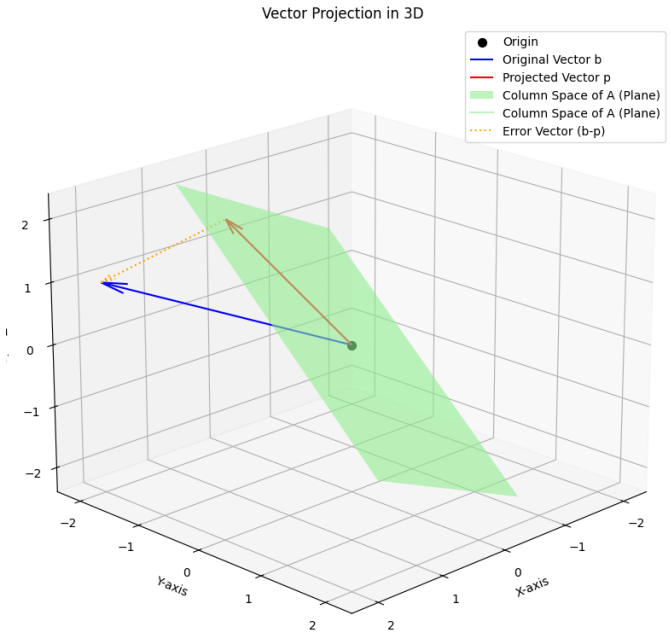

Figure 5:

```
import numpy as np
from scipy.linalg import null_space, orth
from fractions import Fraction
import math
import matplotlib.pyplot as plt # Import matplotlib for plotting

# Helper function to plot vectors and subspaces
def plot_vector(ax, origin, vector, color, linestyle, label):
    if len(vector) == 2:
        ax.quiver(origin[0], origin[1], vector[0], vector[1],
           angles='xy', scale_units='xy', scale=1, color=color,
           linestyle=linestyle, label=label)
    elif len(vector) == 3:
```

```
        ax.quiver(origin[0], origin[1], origin[2], vector[0],
            vector[1], vector[2], color=color, linestyle=linestyle,
            label=label)

def plot_subspace(ax, basis_vectors, color, alpha, label, limits):
    if not basis_vectors:
        # Plot a point at origin for zero-dimensional subspace
        if len(limits) == 4: # 2D
            ax.plot(0, 0, 'o', color=color, markersize=5,
                label=label + ' (origin)')
        elif len(limits) == 6: # 3D
            ax.plot([0], [0], [0], 'o', color=color, markersize=5,
                label=label + ' (origin)')
        return

    dim = len(basis_vectors[0])

    if dim == 2:
        if len(basis_vectors) == 1:
            # Line through origin
            vec = basis_vectors[0]
            if np.allclose(vec, 0):
                ax.plot(0, 0, 'o', color=color, markersize=5,
                    label=label + ' (origin)')
                return

            # Calculate points for the line based on limits
            max_coord = max(abs(limits[0]), abs(limits[1]),
                abs(limits[2]), abs(limits[3]))
            t = np.linspace(-2 * max_coord, 2 * max_coord, 100)
            line_x = t * vec[0]
            line_y = t * vec[1]
            ax.plot(line_x, line_y, color=color, alpha=alpha,
                label=label)
        elif len(basis_vectors) == 2 and
            np.linalg.matrix_rank(basis_vectors) == 2:
            # Spans R^2, fill the whole area (conceptually)
            # No need to explicitly plot a filled area, the axes
                represent it.
            pass # Implied by the axes limits
        else:
            # Multiple vectors, but maybe not full rank or more
                than 2, still lines
            for vec in basis_vectors:
                if not np.allclose(vec, 0):
                    max_coord = max(abs(limits[0]), abs(limits[1]),
                        abs(limits[2]), abs(limits[3]))
```

```
                t = np.linspace(-2 * max_coord, 2 * max_coord,
                    100)
                line_x = t * vec[0]
                line_y = t * vec[1]
                ax.plot(line_x, line_y, color=color,
                    alpha=alpha, linestyle=':', label=f'{label}
                    Component')

    elif dim == 3:
        if len(basis_vectors) == 1:
            # Line through origin
            vec = basis_vectors[0]
            if np.allclose(vec, 0):
                ax.plot([0], [0], [0], 'o', color=color,
                    markersize=5, label=label + ' (origin)')
                return

            max_coord = max(abs(limits[0]), abs(limits[1]),
                abs(limits[2]), abs(limits[3]), abs(limits[4]),
                abs(limits[5]))
            t = np.linspace(-2 * max_coord, 2 * max_coord, 100)
            line_x = t * vec[0]
            line_y = t * vec[1]
            line_z = t * vec[2]
            ax.plot(line_x, line_y, line_z, color=color,
                alpha=alpha, label=label)

        elif len(basis_vectors) == 2 and
            np.linalg.matrix_rank(basis_vectors) == 2:
            # Plane through origin
            v1, v2 = basis_vectors[0], basis_vectors[1]
            if np.allclose(v1, 0) and np.allclose(v2, 0):
                ax.plot([0], [0], [0], 'o', color=color,
                    markersize=5, label=label + ' (origin)')
                return

            s = np.linspace(-1, 1, 10)
            t = np.linspace(-1, 1, 10)
            S, T = np.meshgrid(s, t)
            plane_x = S * v1[0] + T * v2[0]
            plane_y = S * v1[1] + T * v2[1]
            plane_z = S * v1[2] + T * v2[2]
            ax.plot_surface(plane_x, plane_y, plane_z, color=color,
                alpha=alpha, rstride=1, cstride=1, label=label)

        elif len(basis_vectors) == 3 and
            np.linalg.matrix_rank(basis_vectors) == 3:
```

```
            # Spans R^3, fill the whole volume (conceptually)
            pass # Implied by the axes limits
        else:
            # Multiple vectors, but maybe not full rank or more
               than 3, still lines or plane
            for vec in basis_vectors:
                if not np.allclose(vec, 0):
                    max_coord = max(abs(limits[0]), abs(limits[1]),
                       abs(limits[2]), abs(limits[3]),
                       abs(limits[4]), abs(limits[5]))
                    t = np.linspace(-2 * max_coord, 2 * max_coord,
                       100)
                    line_x = t * vec[0]
                    line_y = t * vec[1]
                    line_z = t * vec[2]
                    ax.plot(line_x, line_y, line_z, color=color,
                       alpha=alpha, linestyle=':', label=f'{label}
                       Component')

def get_matrix_input():
    """Prompts the user to enter matrix dimensions and elements."""
    while True:
        try:
            rows = int(input("Enter the number of rows: "))
            cols = int(input("Enter the number of columns: "))
            if rows <= 0 or cols <= 0:
                print("Dimensions must be positive integers. Please
                   try again.")
                continue
            break
        except ValueError:
            print("Invalid input. Please enter integers for
               dimensions.")

    matrix_elements = []
    print(f"Enter the elements of the {rows}x{cols} matrix row by
       row:")
    for i in range(rows):
        while True:
            try:
                row_str = input(f"Enter elements for row {i + 1}
                   (space-separated): ")
                row = list(map(float, row_str.split()))
                if len(row) != cols:
                    print(f"Expected {cols} elements, but got
                       {len(row)}. Please try again.")
                    continue
```

```
                matrix_elements.append(row)
                break
            except ValueError:
                print("Invalid input. Please enter numbers
                    separated by spaces.")

    return np.array(matrix_elements)

def rref(matrix, tol=1e-9):
    """
    Computes the Reduced Row Echelon Form (RREF) of a matrix.
    Uses a robust numerical approach.
    """
    mat = matrix.astype(float)
    rows, cols = mat.shape
    lead = 0
    for r in range(rows):
        if lead >= cols:
            break
        i = r
        while i < rows and abs(mat[i, lead]) < tol:
            i += 1
        if i == rows:
            lead += 1
            continue

        # Swap rows if necessary
        if i != r:
            mat[[i, r]] = mat[[r, i]]

        # Scale row to make pivot 1
        lv = mat[r, lead]
        mat[r, :] = mat[r, :] / lv

        # Eliminate other entries in pivot column
        for i in range(rows):
            if i != r:
                lv = mat[i, lead]
                mat[i, :] = mat[i, :] - lv * mat[r, :]
        lead += 1

    # Set tiny values to zero for cleaner output
    mat[np.abs(mat) < tol] = 0.0
    return mat

def get_null_space_basis_from_rref(matrix, tol=1e-9):
    """
```

```
    Derives a basis for the null space from the RREF of the matrix.
    The basis vectors will typically have integer or simple
       fractional components.
    """
    rref_mat = rref(matrix, tol)
    rows, cols = rref_mat.shape

    pivot_columns = []
    free_variables_indices = []

    # Identify pivot columns and free variables
    current_pivot_row = 0
    for c in range(cols):
        if current_pivot_row < rows and
           abs(rref_mat[current_pivot_row, c] - 1.0) < tol:
            # Check if it's truly a pivot (leading 1, rest of
               column is zero)
            is_pivot = True
            for r_check in range(rows):
                if r_check != current_pivot_row and
                   abs(rref_mat[r_check, c]) > tol:
                    is_pivot = False
                    break
            if is_pivot:
                pivot_columns.append(c)
                current_pivot_row += 1
            else:
                free_variables_indices.append(c)
        else:
            free_variables_indices.append(c)

    if not free_variables_indices:
        return [] # Only trivial solution (zero vector)

    null_space_basis = []
    for free_var_idx in free_variables_indices:
        # Create a vector for this free variable
        basis_vector = np.zeros(cols)
        basis_vector[free_var_idx] = 1 # Set this free variable to 1

        # Express pivot variables in terms of free variables
        for p_row, p_col in enumerate(pivot_columns):
            # Ensure p_row is within the bounds of rref_mat's rows
            if p_row < rows:
                basis_vector[p_col] = -rref_mat[p_row, free_var_idx]

        # Attempt to convert to integers or simpler fractions
```

```
        # Find a common denominator if fractions are present
        temp_vec = []
        denominators = [1]
        for val in basis_vector:
            f = Fraction(val).limit_denominator(1000) # Limit
               denominator to avoid huge numbers
            temp_vec.append(f)
            denominators.append(f.denominator)

        lcm_denom = 1
        if denominators:
            for d in denominators:
                lcm_denom = math.lcm(lcm_denom, d)

        # Scale by LCM to get integer components if possible
        scaled_vec = np.array([int(round(f.numerator * (lcm_denom /
           f.denominator))) for f in temp_vec])

        # If all elements are nearly integers after scaling, use
           the scaled version
        if np.all(np.abs(scaled_vec - (basis_vector * lcm_denom)) <
           tol * lcm_denom):
            null_space_basis.append(scaled_vec)
        else:
            null_space_basis.append(basis_vector)

    return null_space_basis

# Get matrix input from the user
A = get_matrix_input()
print("\nYour entered matrix A:")
print(A)

print("\n--- Four Fundamental Subspaces ---")

# 1. Column Space of A (C(A))
# Find a basis for the column space using a subset of the original
   columns of A.
column_space_original_basis_list = []
current_basis_stack = None

if A.shape[0] > 0 and A.shape[1] > 0: # Ensure matrix is not empty
    rank_A = np.linalg.matrix_rank(A) # Get the rank of the matrix A
    for j in range(A.shape[1]):
        if len(column_space_original_basis_list) == rank_A: #
           Optimization: stop if basis is full
            break
```

```
        current_column = A[:, j].reshape(-1, 1) # Get the j-th
           column as a column vector

        if current_basis_stack is None: # First column consideration
            if np.any(current_column): # If it's not a zero vector,
               it's linearly independent
                current_basis_stack = current_column
                column_space_original_basis_list.append(current_column.flatten(
        else:
            # Check if current_column is linearly independent of
               columns in current_basis_stack
            # Compare ranks: if adding the column increases rank,
               it's independent
            temp_stack = np.hstack((current_basis_stack,
               current_column))
            if np.linalg.matrix_rank(temp_stack) >
               np.linalg.matrix_rank(current_basis_stack):
                current_basis_stack = temp_stack
                column_space_original_basis_list.append(current_column.flatten(

print("\n1. Basis for the Column Space (C(A)) (using original
   columns):")
if not column_space_original_basis_list:
    print("  Basis is empty (zero vector space).")
else:
    for i, col_vec in enumerate(column_space_original_basis_list):
        print(f"  Vector {i+1}: {col_vec}")

# 2. Null Space of Transpose of A (N(A^T))
# A simplified basis for the null space of A.T derived from RREF.
null_space_at_basis_simplified = get_null_space_basis_from_rref(A.T)
print("\n2. Basis for the Null Space of A Transpose (N(A^T))
   (RREF-derived basis):")
if not null_space_at_basis_simplified:
    print("  Basis is empty (only the zero vector).")
else:
    for i, col in enumerate(null_space_at_basis_simplified):
        print(f"  Vector {i+1}: {col}")

# 3. Row Space of A (C(A^T))
# Find a basis for the row space using a subset of the original
   rows of A.
row_space_original_basis_list = []
current_row_basis_stack = None

if A.shape[0] > 0 and A.shape[1] > 0: # Ensure matrix is not empty
```

```
    rank_A = np.linalg.matrix_rank(A) # Rank of A is the same as
       rank of A.T, which is the dimension of the row space
    for i in range(A.shape[0]):
        if len(row_space_original_basis_list) == rank_A: #
           Optimization: stop if basis is full
            break

        current_row = A[i, :].reshape(1, -1) # Get the i-th row as
           a row vector

        if current_row_basis_stack is None: # First row
           consideration
            if np.any(current_row): # If it's not a zero vector,
               it's linearly independent
                current_row_basis_stack = current_row
                row_space_original_basis_list.append(current_row.flatten())
        else:
            # Check if current_row is linearly independent of rows
               in current_row_basis_stack
            # Compare ranks: if adding the row increases rank, it's
               independent
            temp_stack = np.vstack((current_row_basis_stack,
               current_row))
            if np.linalg.matrix_rank(temp_stack) >
               np.linalg.matrix_rank(current_row_basis_stack):
                current_row_basis_stack = temp_stack
                row_space_original_basis_list.append(current_row.flatten())

print("\n3. Basis for the Row Space (C(A^T)) (using original
   rows):")
if not row_space_original_basis_list:
    print("  Basis is empty (zero vector space).")
else:
    for i, row_vec in enumerate(row_space_original_basis_list):
        print(f"  Vector {i+1}: {row_vec}")

# 4. Null Space of A (N(A))
# A simplified basis for the null space of A derived from RREF.
null_space_a_basis_simplified = get_null_space_basis_from_rref(A)
print("\n4. Basis for the Null Space (N(A)) (RREF-derived basis):")
if not null_space_a_basis_simplified:
    print("  Basis is empty (only the zero vector).")
else:
    for i, col in enumerate(null_space_a_basis_simplified):
        print(f"  Vector {i+1}: {col}")
m, n = A.shape
```

```
def setup_plot(dim, title):
    fig = plt.figure(figsize=(8, 8))
    if dim == 3:
        ax = fig.add_subplot(111, projection='3d')
        ax.set_xlabel('X-axis')
        ax.set_ylabel('Y-axis')
        ax.set_zlabel('Z-axis')
        plot_limits = [-5, 5, -5, 5, -5, 5]
    elif dim == 2:
        ax = fig.add_subplot(111)
        ax.set_aspect('equal', adjustable='box')
        ax.set_xlabel('X-axis')
        ax.set_ylabel('Y-axis')
        plot_limits = [-5, 5, -5, 5]
    else:
        print(f"Plotting for dimension {dim} is not supported.")
        return None, None

    ax.set_title(title)
    ax.grid(True)
    ax.axhline(0, color='black', linewidth=0.5)
    ax.axvline(0, color='black', linewidth=0.5)
    if dim == 3: # Draw a 3D origin point
        ax.scatter(0, 0, 0, color='black', s=50, zorder=10)
    else: # Draw a 2D origin point
        ax.scatter(0, 0, color='black', s=50, zorder=10)

    return ax, plot_limits

# --- Combined Plot for C(A) and N(A^T) (lives in R^m) ---
if m in [2, 3]:
    ax_m, limits_m = setup_plot(m, f'Column Space C(A) and Null
       Space N(A^T) (in R^{m})')
    if ax_m:
        # Plot C(A)
        plot_subspace(ax_m, column_space_original_basis_list,
           color='blue', alpha=0.3, label='C(A) Subspace',
           limits=limits_m)
        for i, vec in enumerate(column_space_original_basis_list):
            plot_vector(ax_m, np.zeros(m), vec, color='navy',
               linestyle='-', label=f'C(A) Basis Vector {i+1}')

        # Plot N(A^T)
        plot_subspace(ax_m, null_space_at_basis_simplified,
           color='orange', alpha=0.3, label='N(A^T) Subspace',
           limits=limits_m)
        for i, vec in enumerate(null_space_at_basis_simplified):
```

```
            plot_vector(ax_m, np.zeros(m), vec, color='darkorange',
                linestyle='--', label=f'N(A^T) Basis Vector {i+1}')

        ax_m.legend()
        if m == 2:
            ax_m.set_xlim(limits_m[0], limits_m[1])
            ax_m.set_ylim(limits_m[2], limits_m[3])
        elif m == 3:
            ax_m.set_xlim(limits_m[0], limits_m[1])
            ax_m.set_ylim(limits_m[2], limits_m[3])
            ax_m.set_zlim(limits_m[4], limits_m[5])
        plt.show()
else:
    print(f"Column Space C(A) and Null Space N(A^T) are in R^{m}.
        Visualization not supported for m > 3.")

# --- Combined Plot for C(A^T) and N(A) (lives in R^n) ---
if n in [2, 3]:
    ax_n, limits_n = setup_plot(n, f'Row Space C(A^T) and Null
        Space N(A) (in R^{n})')
    if ax_n:
        # Plot C(A^T)
        plot_subspace(ax_n, row_space_original_basis_list,
            color='green', alpha=0.3, label='C(A^T) Subspace',
            limits=limits_n)
        for i, vec in enumerate(row_space_original_basis_list):
            plot_vector(ax_n, np.zeros(n), vec, color='darkgreen',
                linestyle='-', label=f'C(A^T) Basis Vector {i+1}')

        # Plot N(A)
        plot_subspace(ax_n, null_space_a_basis_simplified,
            color='red', alpha=0.3, label='N(A) Subspace',
            limits=limits_n)
        for i, vec in enumerate(null_space_a_basis_simplified):
            plot_vector(ax_n, np.zeros(n), vec, color='darkred',
                linestyle='--', label=f'N(A) Basis Vector {i+1}')

        ax_n.legend()
        if n == 2:
            ax_n.set_xlim(limits_n[0], limits_n[1])
            ax_n.set_ylim(limits_n[2], limits_n[3])
        elif n == 3:
            ax_n.set_xlim(limits_n[0], limits_n[1])
            ax_n.set_ylim(limits_n[2], limits_n[3])
            ax_n.set_zlim(limits_n[4], limits_n[5])
        plt.show()
else:
```

```
    print(f"Row Space C(A^T) and Null Space N(A) are in R^{n}.
       Visualization not supported for n > 3.")
```

Problem: Let

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 0 \end{bmatrix}.$$

Find bases for the four fundamental subspaces:

$$\mathcal{R}(A), \quad \mathcal{N}(A), \quad \mathcal{R}(A^T), \quad \mathcal{N}(A^T).$$

OUTPUT

Enter the number of rows: 2
Enter the number of columns: 3
Enter the elements of the 2x3 matrix row by row:
Enter elements for row 1 (space-separated): 1 2 3
Enter elements for row 2 (space-separated): 0 1 0

Your entered matrix A:
[[1. 2. 3.]
[0. 1. 0.]]

— Four Fundamental Subspaces —

1. Basis for the Column Space (C(A)) (using original columns):
Vector 1: [1. 0.]
Vector 2: [2. 1.]

2. Basis for the Null Space of A Transpose (N($A\hat{T}$)) (RREF-derived basis):
Basis is empty (only the zero vector).

3. Basis for the Row Space (C($A\hat{T}$)) (using original rows):
Vector 1: [1. 2. 3.]
Vector 2: [0. 1. 0.]

4. Basis for the Null Space (N(A)) (RREF-derived basis):
Vector 1: [-3 0 1]

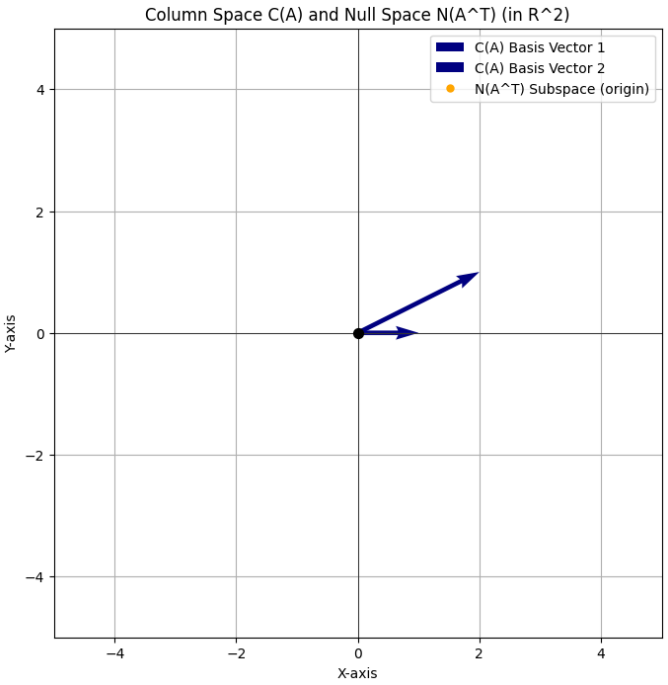

Figure 6:

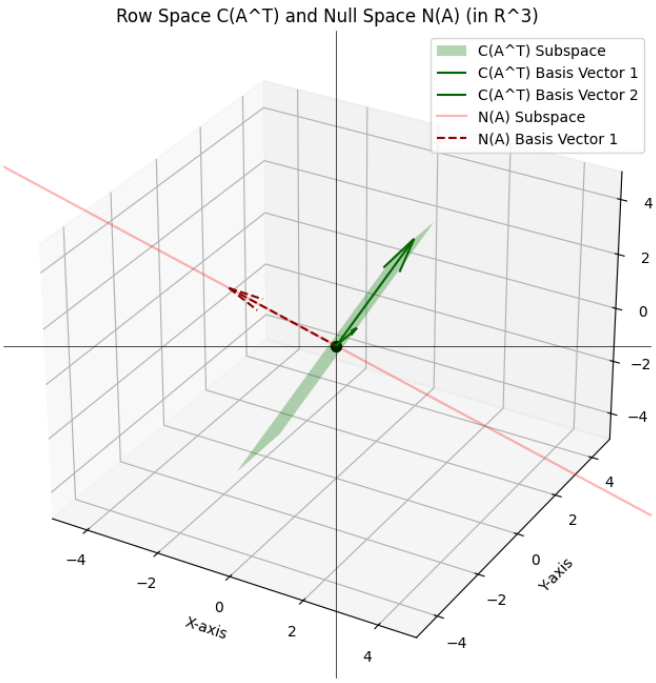

Figure 7:

4 Least Squares Approximation and Inconsistent Systems

Inconsistent Linear Systems

Consider a system of linear equations

$$Ax = b,$$

where A is an $m \times n$ matrix, $x \in \mathbb{R}^n$ is the unknown vector, and $b \in \mathbb{R}^m$ is a given vector.

The system is said to be **consistent** if there exists a solution x satisfying $Ax = b$. Otherwise, it is called **inconsistent**. An inconsistent system has no exact solution because b does not lie in the column space of A.

Idea of Least Squares Approximation

If the system $Ax = b$ is inconsistent, we seek a vector x such that Ax is as close as possible to b. This is done by minimizing the error (or residual)

$$r = b - Ax.$$

We aim to find x that minimizes the squared norm of the residual:

$$\|Ax - b\|^2.$$

This method is called the **least squares method**, and the resulting solution is called the **least squares solution**.

Geometric Interpretation

The vector Ax lies in the column space of A. For an inconsistent system, b does not lie in this space. Hence we find the vector $\hat{b} = Ax$ in the column space of A that is closest to b.

Thus, $\hat{b}$ is the orthogonal projection of b onto $\mathrm{Col}(A)$. The residual vector

$$r = b - Ax$$

is orthogonal to the column space of A.

Normal Equations

Since the residual is orthogonal to every column of A, we obtain

$$A^T(b - Ax) = 0.$$

This gives the **normal equations**:

$$A^T Ax = A^T b.$$

If $A^T A$ is invertible, the least squares solution is

$$x = (A^T A)^{-1} A^T b.$$

Least Squares Regression (Line Fitting)

Suppose we are given data points

$$(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)$$

and we want to fit a straight line

$$y = a + bx.$$

This leads to the system

$$\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_m \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

This system may be inconsistent, so we solve it using least squares:

$$A^T A \begin{bmatrix} a \\ b \end{bmatrix} = A^T b.$$

The resulting values of a and b give the best-fit line that minimizes the sum of squared errors between observed and predicted values.

Listing 1: Least Squares Regression on Log–Log Data

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import linregress
n_points = int(input("Enter the number of data points you wish to
   input: "))
x_var_name = input("Enter the desired name for the x-variable: ")
y_var_name = input("Enter the desired name for the y-variable: ")

x_data = []
y_data = []

for i in range(n_points):
    while True:
        try:
            x_val = float(input(f"Enter {x_var_name} value for
               point {i + 1}: "))
            x_data.append(x_val)
            break
        except ValueError:
            print("Invalid input. Please enter a numerical value
               for x.")

    while True:
        try:
```

```
            y_val = float(input(f"Enter {y_var_name} value for
                point {i + 1}: "))
            y_data.append(y_val)
            break
        except ValueError:
            print("Invalid input. Please enter a numerical value
                for y.")

print(f"\nCollected {n_points} data points:")
print(f"{x_var_name}: {x_data}")
print(f"{y_var_name}: {y_data}")
# Convert lists to NumPy arrays
x_array = np.array(x_data)
y_array = np.array(y_data)

# Identify non-positive values
non_positive_x_mask = x_array <= 0
non_positive_y_mask = y_array <= 0

# Check for and warn about non-positive values
if np.any(non_positive_x_mask) or np.any(non_positive_y_mask):
    print("Warning: Some x or y values are non-positive (<= 0) and
        cannot be used for logarithmic transformation. These points
        will be excluded.")

# Filter out non-positive values
positive_x_mask = x_array > 0
positive_y_mask = y_array > 0

# Combine masks to ensure only points where both x and y are
    positive are kept
combined_positive_mask = positive_x_mask & positive_y_mask

positive_x_array = x_array[combined_positive_mask]
positive_y_array = y_array[combined_positive_mask]

# Calculate the natural logarithm
log_x = np.log(positive_x_array)
log_y = np.log(positive_y_array)

print(f"\nOriginal {x_var_name} values (NumPy array): {x_array}")
print(f"Original {y_var_name} values (NumPy array): {y_array}")
print(f"\nPositive {x_var_name} values for log transform:
    {positive_x_array}")
print(f"Positive {y_var_name} values for log transform:
    {positive_y_array}")
print(f"\nNatural logarithm of {x_var_name}: {log_x}")
```

```
print(f"Natural logarithm of {y_var_name}: {log_y}")

# Plot of original data
plt.figure(figsize=(8, 6))
plt.scatter(x_array, y_array, alpha=0.7, label='Original Data
   Points')
plt.xlabel(x_var_name)
plt.ylabel(y_var_name)
plt.title(f"Original Data: {y_var_name} vs. {x_var_name}")
plt.grid(True)
plt.legend()
plt.show()


plt.figure(figsize=(8, 6))
plt.scatter(log_x, log_y, alpha=0.7, label='Data Points')
plt.xlabel(f"Log of {x_var_name}")
plt.ylabel(f"Log of {y_var_name}")
plt.title(f"Log-Log Plot of {y_var_name} vs. {x_var_name}")
plt.grid(True)

# Perform linear regression on log_x and log_y
slope, intercept, r_value, p_value, std_err = linregress(log_x,
   log_y)

# Generate points for the best-fit line
log_x_fit = np.linspace(min(log_x), max(log_x), 100)
log_y_fit = slope * log_x_fit + intercept

# Plot the best-fit line
plt.plot(log_x_fit, log_y_fit, color='red', linestyle='--',
   label='Best-Fit Line')
plt.legend()
plt.show()

print(f"Regression Results for Log-Log Data:")
print(f"  Slope (b): {slope:.4f}")
print(f"  Intercept (log(a)): {intercept:.4f}")
print(f"  R-value: {r_value:.4f}")
print(f"  P-value: {p_value:.4f}")
print(f"  Standard Error: {std_err:.4f}")

# Print the equation of the line
print(f"\nEquation of the Best-Fit Line: log({y_var_name}) =
   {slope:.4f} * log({x_var_name}) + {intercept:.4f}")
```

Problem: Consider the following data representing the distance x (in astronomical units) of planets from the Sun and their orbital period y (in Earth years):

Planet	Distance x	Period y
Mercury	0.387	0.241
Venus	0.723	0.615
Earth	1.000	1.000
Mars	1.523	1.881
Jupiter	5.203	11.861
Saturn	9.541	29.450

Find the least squares fit for the given data.

OUTPUT

Enter the number of data points you wish to input: 6
Enter the desired name for the x-variable: distance
Enter the desired name for the y-variable: period
Enter distance value for point 1: 0.387
Enter period value for point 1: 0.241
Enter distance value for point 2: 0.723
Enter period value for point 2: 0.615
Enter distance value for point 3: 1
Enter period value for point 3: 1
Enter distance value for point 4: 1.523
Enter period value for point 4: 1.881
Enter distance value for point 5: 5.203
Enter period value for point 5: 11.861
Enter distance value for point 6: 9.541
Enter period value for point 6: 29.457

Collected 6 data points:
distance: [0.387, 0.723, 1.0, 1.523, 5.203, 9.541]
period: [0.241, 0.615, 1.0, 1.881, 11.861, 29.457]

Original distance values (NumPy array): [0.387 0.723 1. 1.523 5.203 9.541]
Original period values (NumPy array): [0.241 0.615 1. 1.881 11.861 29.457]

Positive distance values for log transform: [0.387 0.723 1. 1.523 5.203 9.541]
Positive period values for log transform: [0.241 0.615 1. 1.881 11.861 29.457]

Natural logarithm of distance: [-0.94933059 -0.32434606 0. 0.42068207 1.64923538 2.2555983
]
Natural logarithm of period: [-1.42295835 -0.48613301 0. 0.63180355 2.47325571 3.38293157]

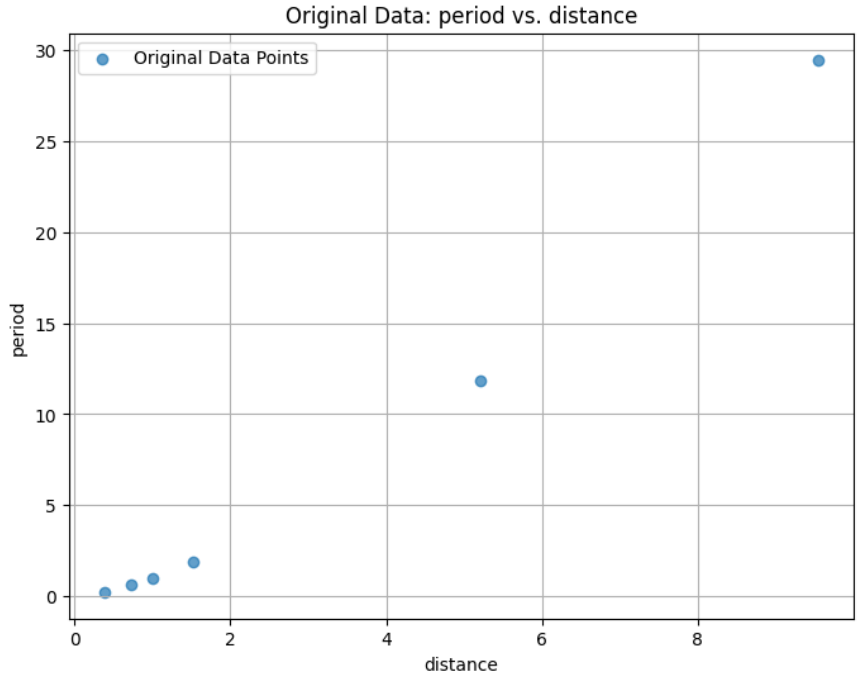

Figure 8:

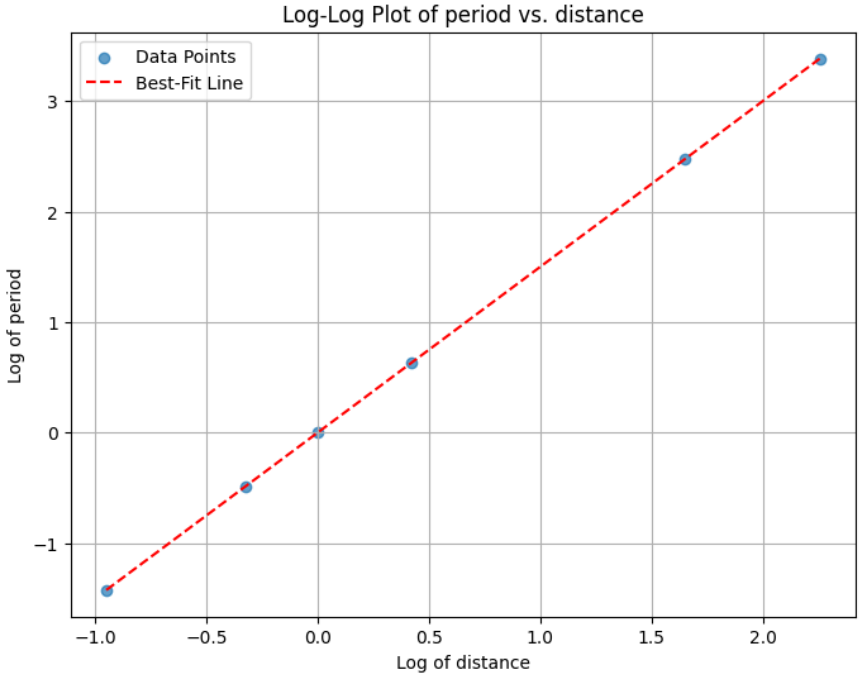

Figure 9:

Regression Results for Log-Log Data:
Slope (b): 1.4995

Intercept (log(a)): 0.0004
R-value: 1.0000
P-value: 0.0000
Standard Error: 0.0001

Equation of the Best-Fit Line: log(period) = 1.4995 * log(distance) + 0.0004

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 1. Input Matrix A and Vector b
# Prompt the user to enter the number of rows for matrix A.
rows_A = int(input("Enter the number of rows for matrix A: "))
print(f"Number of rows for Matrix A: {rows_A}")

cols_A = int(input("Enter the number of columns for matrix A: "))
print(f"Number of columns for Matrix A: {cols_A}")

matrix_A_elements = []
print("\nEnter the elements for matrix A (row by row):")
for i in range(rows_A):
    row = []
    for j in range(cols_A):
        while True:
            try:
                element = float(input(f"Enter element
                   A[{i+1}][{j+1}]: "))
                row.append(element)
                break
            except ValueError:
                print("Invalid input. Please enter a numerical
                   value.")
    matrix_A_elements.append(row)

# Convert the list of lists to a NumPy array
matrix_A = np.array(matrix_A_elements)
print("\nMatrix A:\n", matrix_A)

while True:
    try:
        b_input = input(f"\nEnter the elements for vector b
           (space-separated, {rows_A} elements expected): ")
        b_elements = [float(x) for x in b_input.split()]
        vector_b = np.array(b_elements)

        if vector_b.shape[0] == rows_A:
```

```
            print("\nVector b:\n", vector_b)
            break
        else:
            print(f"Dimension mismatch: Vector b must have {rows_A}
               elements, but {vector_b.shape[0]} were entered.
               Please re-enter.")
    except ValueError:
        print("Invalid input. Please enter numerical values
           separated by spaces.")


# 2. Calculate the least squares solution x_hat
x_hat, residuals, rank, s = np.linalg.lstsq(matrix_A, vector_b,
   rcond=None)

print("\nLeast Squares Solution (x_hat):")
print(x_hat)


# 3. Calculate the projection p and error vector e
projection_p = matrix_A @ x_hat
error_e = vector_b - projection_p

print("\nProjection of b onto the column space of A (p):")
print(projection_p)

print("\nError vector (e = b - p):")
print(error_e)


# 4. Visualize the column space, b, p, and e in 3D
# Get the column vectors of A
col_1 = matrix_A[:, 0]
col_2 = matrix_A[:, 1]

# Generate points to visualize the column space (a plane in 3D)
s1 = np.linspace(-3, 3, 10)
s2 = np.linspace(-3, 3, 10)
S1, S2 = np.meshgrid(s1, s2)

# Points on the plane formed by the column space of A
x_plane = S1 * col_1[0] + S2 * col_2[0]
y_plane = S1 * col_1[1] + S2 * col_2[1]
z_plane = S1 * col_1[2] + S2 * col_2[2]

# Create the 3D plot
fig = plt.figure(figsize=(10, 8))
```

```
ax = fig.add_subplot(111, projection='3d')

# Plot the column space of A as a plane
ax.plot_surface(x_plane, y_plane, z_plane, alpha=0.5, rstride=100,
    cstride=100, color='lightblue', label='Col(A)')

# Plot vector b
ax.quiver(0, 0, 0, vector_b[0], vector_b[1], vector_b[2],
    color='red', linewidth=2, arrow_length_ratio=0.1, label='Vector
    b')

# Plot projection p
ax.quiver(0, 0, 0, projection_p[0], projection_p[1],
    projection_p[2], color='green', linewidth=2,
    arrow_length_ratio=0.1, label='Projection p')

# Plot error vector e
ax.quiver(projection_p[0], projection_p[1], projection_p[2],
    error_e[0], error_e[1], error_e[2], color='purple', linewidth=2,
    arrow_length_ratio=0.1, label='Error e = b - p')

# Plot column vectors of A
ax.quiver(0, 0, 0, col_1[0], col_1[1], col_1[2], color='blue',
    linestyle='--', arrow_length_ratio=0.1, label='Col A1')
ax.quiver(0, 0, 0, col_2[0], col_2[1], col_2[2], color='orange',
    linestyle='--', arrow_length_ratio=0.1, label='Col A2')

# Set labels and title
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_zlabel('Z-axis')
ax.set_title('Least Squares Visualization')

# Set equal aspect ratio for better visualization
# This might not work perfectly with plot_surface and can distort
    perception.
# Instead, manually set limits based on data range if needed.
max_val = np.max(np.abs([vector_b, projection_p, error_e, col_1,
    col_2]))
ax.set_xlim([-max_val, max_val])
ax.set_ylim([-max_val, max_val])
ax.set_zlim([-max_val, max_val])

# Add a legend outside the plot for clarity
ax.legend(loc='center left', bbox_to_anchor=(1.05, 0.5))

plt.grid(True)
```

```
plt.show()


# 5. Summarize the insights
print(f"\nCalculated Least Squares Solution (x_hat): {x_hat}")
print(f"Projection of b onto Col(A) (p): {projection_p}")
print(f"Error Vector (e = b - p): {error_e}")

print("\n### Insights from Visualization:\n")
print("1. The column space of matrix A is represented by the light
   blue plane. This plane contains all possible linear combinations
   of the column vectors of A.")
print("2. The original vector b (red arrow) does not lie in the
   column space of A, indicating that there is no exact solution to
   Ax = b.")
print("3. The projection p (green arrow) is the vector in the
   column space of A that is closest to b. This is the core idea of
   the least squares solution: finding the best possible
   approximation of b within the column space.")
print("4. The error vector e = b - p (purple arrow) is the
   difference between the original vector b and its projection p. A
   key property of least squares is that this error vector is
   orthogonal (perpendicular) to the column space of A. This is
   visually evident as the purple arrow extends from the plane
   directly towards b.")
print("5. The least squares solution x_hat [1. -1.] tells us that
   the projection p is formed by 1 * col_1 + (-1) * col_2.")
print("This visualization clearly demonstrates how the least
   squares method finds the best approximate solution by projecting
   the target vector onto the subspace spanned by the matrix's
   columns, minimizing the error (the length of the error vector).")
```

Problem: Let

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \\ 1 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 2 \\ 0 \\ -3 \end{bmatrix}.$$

Find the least squares solution of the system

$$Ax = b.$$

OUTPUT

Enter the number of rows for matrix A: 3
Number of rows for Matrix A: 3
Enter the number of columns for matrix A: 2

```
Number of columns for Matrix A: 2

    Enter the elements for matrix A (row by row):
Enter element A[1][1]: 2
Enter element A[1][2]: 1
Enter element A[2][1]: 1
Enter element A[2][2]: 2
Enter element A[3][1]: 1
Enter element A[3][2]: 1

    Matrix A:
[[2. 1.]
[1. 2.]
[1. 1.]]

    Enter the elements for vector b (space-separated, 3 elements expected): 2 0 -3

    Vector b:
[ 2. 0. -3.]

    Least Squares Solution (x_hat):
[ 1. -1.]

    Projection of b onto the column space of A (p):
[ 1.00000000e+00 -1.00000000e+00 3.33066907e-16]

    Error vector (e = b - p):
[ 1. 1. -3.]
```

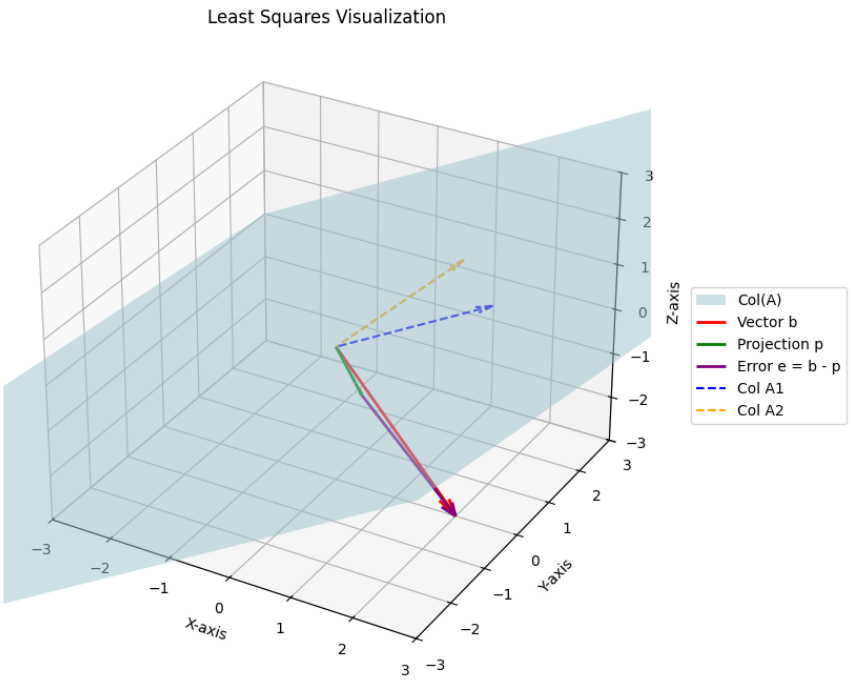

Figure 10:

Calculated Least Squares Solution (x_hat): [1. -1.]
Projection of b onto Col(A) (p): [1.00000000e+00 -1.00000000e+00 3.33066907e-16]
Error Vector (e = b - p): [1. 1. -3.]

Insights from Visualization:

1. The column space of matrix A is represented by the light blue plane. This plane contains all possible linear combinations of the column vectors of A.
2. The original vector b (red arrow) does not lie in the column space of A, indicating that there is no exact solution to Ax = b.
3. The projection p (green arrow) is the vector in the column space of A that is closest to b. This is the core idea of the least squares solution: finding the best possible approximation of b within the column space.
4. The error vector e = b - p (purple arrow) is the difference between the original vector b and its projection p. A key property of least squares is that this error vector is orthogonal (perpendicular) to the column space of A. This is visually evident as the purple arrow extends from the plane directly towards b.
5. The least squares solution x_hat [1. -1.] tells us that the projection p is formed by 1 * col_1 + (-1) * col_2.

This visualization clearly demonstrates how the least squares method finds the best approximate solution by projecting the target vector onto the subspace spanned by the matrix's columns, minimizing the error (the length of the error vector).

5 Cross Product

5.1 Motivation: Why Do We Need the Cross Product?

In $\mathbb{R}^3$, the dot product measures how parallel two vectors are. However, many geometric and physical problems require:

- A vector perpendicular to two given vectors,
- The area of a parallelogram formed by two vectors,
- A direction determined by orientation (right-hand rule),
- Applications in physics such as torque and angular momentum.

The cross product provides exactly such a vector.

5.2 Standard Orthonormal Basis of $\mathbb{R}^3$

The standard orthonormal basis vectors are:

$$\mathbf{i} = (1, 0, 0), \quad \mathbf{j} = (0, 1, 0), \quad \mathbf{k} = (0, 0, 1)$$

They satisfy:

$$\mathbf{i} \cdot \mathbf{i} = \mathbf{j} \cdot \mathbf{j} = \mathbf{k} \cdot \mathbf{k} = 1$$

$$\mathbf{i} \cdot \mathbf{j} = \mathbf{j} \cdot \mathbf{k} = \mathbf{k} \cdot \mathbf{i} = 0$$

Their cross product relations are:

$$\mathbf{i} \times \mathbf{j} = \mathbf{k}, \quad \mathbf{j} \times \mathbf{k} = \mathbf{i}, \quad \mathbf{k} \times \mathbf{i} = \mathbf{j}$$

Reversing order changes sign:

$$\mathbf{j} \times \mathbf{i} = -\mathbf{k}$$

5.3 Definition of the Cross Product

Let

$$\mathbf{a} = (a_1, a_2, a_3), \quad \mathbf{b} = (b_1, b_2, b_3)$$

The cross product $\mathbf{a} \times \mathbf{b}$ is defined as:

$$\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$$

Expanding,

$$\mathbf{a} \times \mathbf{b} = (a_2 b_3 - a_3 b_2)\mathbf{i} - (a_1 b_3 - a_3 b_1)\mathbf{j} + (a_1 b_2 - a_2 b_1)\mathbf{k}$$

5.4 Geometrical Meaning

If θ is the angle between $\mathbf{a}$ and $\mathbf{b}$, then:

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \, \|\mathbf{b}\| \sin\theta$$

Thus:

- The magnitude equals the area of the parallelogram formed by $\mathbf{a}$ and $\mathbf{b}$.
- The direction is perpendicular to both vectors.
- The orientation follows the right-hand rule.

5.5 Area Interpretation

Let $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ be two vectors placed tail-to-tail. Let θ be the angle between them.

From elementary geometry,

$$\text{Area} = (\text{base}) \times (\text{height})$$

Choose $\|\mathbf{a}\|$ as the base. The height is the component of $\mathbf{b}$ perpendicular to $\mathbf{a}$:

$$\text{height} = \|\mathbf{b}\| \sin\theta$$

Thus,

$$\text{Area} = \|\mathbf{a}\| \|\mathbf{b}\| \sin\theta$$

But from the definition of the cross product,

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \|\mathbf{b}\| \sin\theta$$

Therefore,

$$\boxed{\text{Area of parallelogram} = \|\mathbf{a} \times \mathbf{b}\|}$$

- $\mathbf{a} \times \mathbf{b}$ is perpendicular to both $\mathbf{a}$ and $\mathbf{b}$.
- Its norm equals the area of the parallelogram.
- If $\mathbf{a}$ and $\mathbf{b}$ are parallel, then $\sin\theta = 0$, hence

$$\|\mathbf{a} \times \mathbf{b}\| = 0.$$

A diagonal of a parallelogram divides it into two congruent triangles. Hence, each triangle has half the area of the parallelogram.

Therefore,

$$\boxed{\text{Area of triangle} = \frac{1}{2}\|\mathbf{a} \times \mathbf{b}\|}$$

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def get_vector_input(name):
    while True:
        try:
            input_str = input(f"Enter 3 comma-separated values for
               vector {name} (e.g., 1,2,3): ")
            values = [float(x.strip()) for x in
               input_str.split(',')]
            if len(values) == 3:
                return np.array(values)
            else:
                print("Please enter exactly 3 values.")
        except ValueError:
            print("Invalid input. Please enter numbers separated by
               commas.")

# Get input for vectors u and v
u = get_vector_input('u')
v = get_vector_input('v')

print(f"Vector u: {u}")
print(f"Vector v: {v}")

# Calculate u cross v
u_cross_v = np.cross(u, v)
print(f"u cross v: {u_cross_v}")

# Calculate v cross u
v_cross_u = np.cross(v, u)
print(f"v cross u: {v_cross_u}")

# Show antisymmetry
print("\n Demonstrating antisymmetry (u cross v = -(v cross u)):")
print(f"u cross v == -(v cross u): {np.allclose(u_cross_v,
   -v_cross_u)}")

# Calculate u cross u
u_cross_u = np.cross(u, u)
print(f"u cross u: {u_cross_u}")

# Calculate v cross v
v_cross_v = np.cross(v, v)
print(f"v cross v: {v_cross_v}")
```

```
# Show they are zero vectors
print("\nDemonstrating that the cross product of a vector with
   itself is the zero vector:")
print(f"u cross u is a zero vector: {np.allclose(u_cross_u,
   np.array([0, 0, 0]))}")
print(f"v cross v is a zero vector: {np.allclose(v_cross_v,
   np.array([0, 0, 0]))}")

# 3D Plotting
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Function to draw a vector from the origin
def plot_vector(vector, color, label):
    ax.quiver(0, 0, 0, vector[0], vector[1], vector[2],
       color=color, label=label, arrow_length_ratio=0.2)

# Plot the vectors
plot_vector(u, 'r', 'Vector u')
plot_vector(v, 'g', 'Vector v')
plot_vector(u_cross_v, 'b', 'u x v')
plot_vector(v_cross_u, 'm', 'v x u')

# Set limits and labels
max_val = max(np.abs(np.concatenate([u, v, u_cross_v, v_cross_u])))
ax.set_xlim([-max_val*2.5, max_val*2.5])
ax.set_ylim([-max_val*2.5, max_val*2.5])
ax.set_zlim([-max_val*2.5, max_val*2.5])
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_zlabel('Z-axis')
ax.set_title('Vectors and their Cross Products in 3D')

ax.legend()
ax.grid(True)

plt.show()
```

Problem: Let

$$\mathbf{u} = \mathbf{i} - 2\mathbf{j} + \mathbf{k}, \quad \mathbf{v} = 3\mathbf{i} + \mathbf{j} - 2\mathbf{k}.$$

Compute:

$\mathbf{u} \times \mathbf{v}$

$\mathbf{v} \times \mathbf{u}$

$\mathbf{v} \times \mathbf{v}$

OUTPUT

Enter 3 comma-separated values for vector u (e.g., 1,2,3): 1,-2,1
Enter 3 comma-separated values for vector v (e.g., 1,2,3): 3,1,-2
Vector u: [1. -2. 1.]
Vector v: [3. 1. -2.]
u cross v: [3. 5. 7.]
v cross u: [-3. -5. -7.]

Demonstrating antisymmetry (u cross v = -(v cross u)):
u cross v == -(v cross u): True
u cross u: [0. 0. 0.]
v cross v: [0. 0. 0.]

Demonstrating that the cross product of a vector with itself is the zero vector:
u cross u is a zero vector: True
v cross v is a zero vector: True

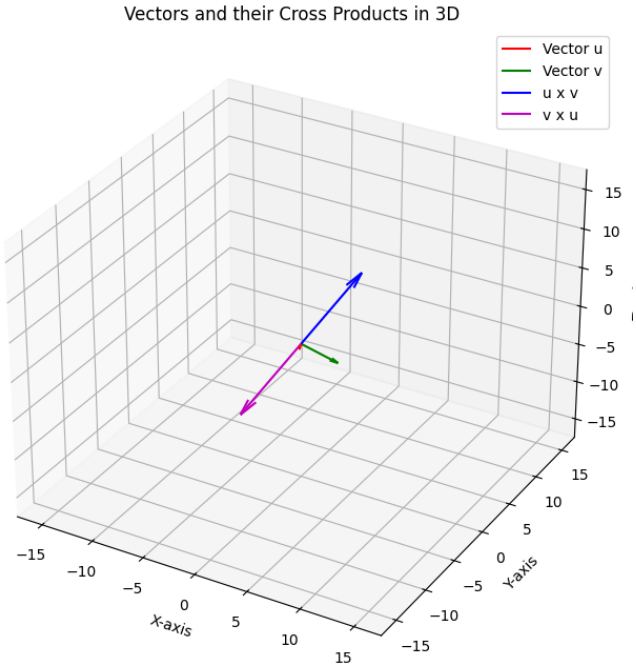

Figure 11:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
```

```
def get_vector_input(name):
    while True:
        try:
            input_str = input(f"Enter 3 comma-separated values for
               vector {name} (e.g., 1,2,3): ")
            values = [float(x.strip()) for x in
               input_str.split(',')]
            if len(values) == 3:
                return np.array(values)
            else:
                print("Please enter exactly 3 values.")
        except ValueError:
            print("Invalid input. Please enter numbers separated by
               commas.")

# Get input for vectors u and v (adjacent sides of the
   parallelogram)
u_parallelogram = get_vector_input('u (adjacent side)')
v_parallelogram = get_vector_input('v (adjacent side)')

print(f"\nVector u: {u_parallelogram}")
print(f"Vector v: {v_parallelogram}")

# Calculate the cross product of u and v
cross_product_parallelogram = np.cross(u_parallelogram,
   v_parallelogram)
print(f"Cross product of u and v: {cross_product_parallelogram}")

# Calculate the magnitude (norm) of the cross product
# The magnitude of the cross product of two vectors is equal to the
   area of the parallelogram
# spanned by the vectors.
area_parallelogram = np.linalg.norm(cross_product_parallelogram)

print(f"\nThe area of the parallelogram formed by vectors u and
   v(norm of the cross product vector) is: {area_parallelogram}")

# Create a figure and a 3D axes object
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Plot vectors u and v as arrows from the origin
# The quiver function takes (origin_x, origin_y, origin_z, end_x,
   end_y, end_z)
ax.quiver(0, 0, 0, u_parallelogram[0], u_parallelogram[1],
   u_parallelogram[2],
          color='r', arrow_length_ratio=0.1, label='Vector u')
```

```
ax.quiver(0, 0, 0, v_parallelogram[0], v_parallelogram[1],
    v_parallelogram[2],
           color='b', arrow_length_ratio=0.1, label='Vector v')

# Define the vertices of the parallelogram
origin = np.array([0, 0, 0])
point_u = u_parallelogram
point_v = v_parallelogram
point_u_plus_v = u_parallelogram + v_parallelogram

# Define the vertices for shading the parallelogram
# The order of vertices matters for correct shading
verts = [
    [origin, point_u, point_u_plus_v, point_v] # This defines the
       four corners of the parallelogram
]

# Create a Poly3DCollection to shade the parallelogram area
poly = Poly3DCollection(verts, alpha=0.3, facecolor='g',
    edgecolor='k')
ax.add_collection3d(poly)

# Plot the edges of the parallelogram (as dashed lines)
ax.plot([0, u_parallelogram[0]], [0, u_parallelogram[1]], [0,
    u_parallelogram[2]], 'r--')
ax.plot([0, v_parallelogram[0]], [0, v_parallelogram[1]], [0,
    v_parallelogram[2]], 'b--')
ax.plot([u_parallelogram[0], point_u_plus_v[0]],
    [u_parallelogram[1], point_u_plus_v[1]], [u_parallelogram[2],
    point_u_plus_v[2]], 'r--')
ax.plot([v_parallelogram[0], point_u_plus_v[0]],
    [v_parallelogram[1], point_u_plus_v[1]], [v_parallelogram[2],
    point_u_plus_v[2]], 'b--')

# Set labels and title for the plot
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_zlabel('Z-axis')
ax.set_title('Parallelogram formed by Vectors u and v')
ax.legend()

# Dynamically set plot limits to encompass all vectors and the
    parallelogram
all_coords = np.concatenate((origin, u_parallelogram,
    v_parallelogram, point_u_plus_v))
max_val = np.max(np.abs(all_coords))
ax.set_xlim([-max_val, max_val])
```

```
ax.set_ylim([-max_val, max_val])
ax.set_zlim([-max_val, max_val])

# Display the plot
plt.show()
```

Problem: Let

$$\mathbf{u} = -3\mathbf{i} + 4\mathbf{j} + \mathbf{k}, \quad \mathbf{v} = -2\mathbf{j} + 6\mathbf{k}.$$

Find the area of the parallelogram formed by the vectors.

OUTPUT

Enter 3 comma-separated values for vector u (adjacent side) (e.g., 1,2,3): -3,4,1
Enter 3 comma-separated values for vector v (adjacent side) (e.g., 1,2,3): 0,-2,6

Vector u: [-3. 4. 1.]
Vector v: [0. -2. 6.]
Cross product of u and v: [26. 18. 6.]

The area of the parallelogram formed by vectors u and v(norm of the cross product vector) is: 32.18695387886216

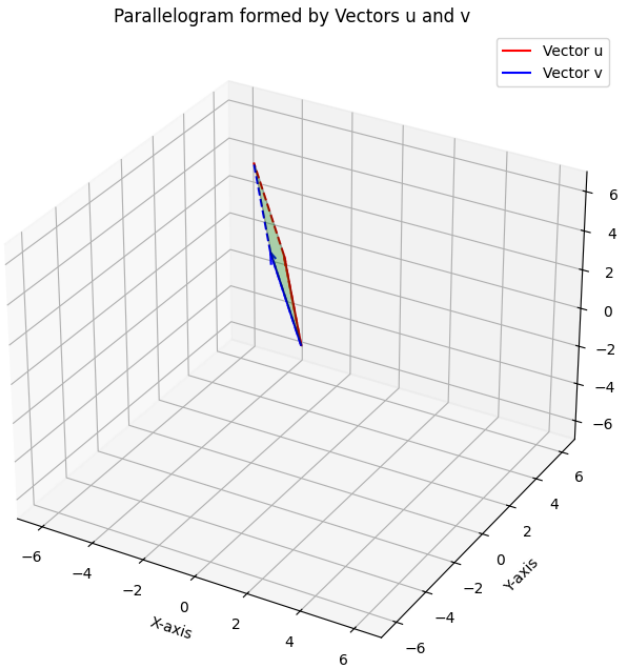

Figure 12:

5.6 Properties of the Cross Product

Let $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$ and let $\lambda \in \mathbb{R}$.

1. Anti-Commutativity

$$\mathbf{a} \times \mathbf{b} = -\mathbf{b} \times \mathbf{a}$$

Proof:
Using the determinant definition,

$$\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$$

Interchanging two rows of a determinant changes its sign. Hence,

$$\mathbf{b} \times \mathbf{a} = -\mathbf{a} \times \mathbf{b}.$$

Geometric Meaning:
Reversing the order reverses orientation. The magnitude remains the same, but the direction flips (right-hand rule).

2. Self Cross Product

$$\mathbf{a} \times \mathbf{a} = \mathbf{0}$$

Proof:

$$\|\mathbf{a} \times \mathbf{a}\| = \|\mathbf{a}\|\|\mathbf{a}\| \sin 0 = 0.$$

Hence the vector must be $\mathbf{0}$.
Geometric Meaning:
A vector cannot generate area with itself since no parallelogram is formed.

3. Distributivity Over Addition

$$\mathbf{a} \times (\mathbf{b} + \mathbf{c}) = \mathbf{a} \times \mathbf{b} + \mathbf{a} \times \mathbf{c}$$

Proof:
Using determinant linearity in rows,

$$\begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 + c_1 & b_2 + c_2 & b_3 + c_3 \end{vmatrix} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} + \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ c_1 & c_2 & c_3 \end{vmatrix}.$$

Geometric Meaning:
Area behaves linearly — combining vectors combines the corresponding oriented areas.

4. Scalar Multiplication

$$(\lambda \mathbf{a}) \times \mathbf{b} = \lambda(\mathbf{a} \times \mathbf{b})$$

Proof:
Pull λ outside the determinant:

$$\begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ \lambda a_1 & \lambda a_2 & \lambda a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = \lambda \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}.$$

Geometric Meaning:
Scaling one vector scales the area proportionally.

5. Orthogonality

$$\mathbf{a} \cdot (\mathbf{a} \times \mathbf{b}) = 0$$

Let $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$.
We know that the cross product $\mathbf{a} \times \mathbf{b}$ is perpendicular to both $\mathbf{a}$ and $\mathbf{b}$.
Hence,

$$\mathbf{a} \perp (\mathbf{a} \times \mathbf{b}).$$

Now, recall that if two vectors $\mathbf{u}$ and $\mathbf{v}$ are perpendicular, then

$$\mathbf{u} \cdot \mathbf{v} = 0.$$

Applying this property here, we obtain

$$\mathbf{a} \cdot (\mathbf{a} \times \mathbf{b}) = 0.$$

Similarly, since $\mathbf{b}$ is also perpendicular to $\mathbf{a} \times \mathbf{b}$, we have

$$\mathbf{b} \cdot (\mathbf{a} \times \mathbf{b}) = 0.$$

Geometrical Meaning:
The vector $\mathbf{a} \times \mathbf{b}$ represents a vector normal to the plane containing $\mathbf{a}$ and $\mathbf{b}$. Since it is orthogonal to this plane, its dot product with any vector lying in the plane (such as $\mathbf{a}$ or $\mathbf{b}$) must be zero.

6.Scalar Triple Product

Definition.
Let $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$. The scalar triple product is defined as

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}).$$

It is a scalar quantity.

Determinant Form.
If

$$\mathbf{a} = (a_1, a_2, a_3), \quad \mathbf{b} = (b_1, b_2, b_3), \quad \mathbf{c} = (c_1, c_2, c_3),$$

then

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \det \begin{pmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{pmatrix}.$$

Proof.
First compute the cross product:

$$\mathbf{b} \times \mathbf{c} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{vmatrix}.$$

Expanding along the first row,

$$\mathbf{b} \times \mathbf{c} = (b_2c_3 - b_3c_2,\ -(b_1c_3 - b_3c_1),\ b_1c_2 - b_2c_1).$$

Now taking dot product with $\mathbf{a}$,

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = a_1(b_2c_3 - b_3c_2) - a_2(b_1c_3 - b_3c_1) + a_3(b_1c_2 - b_2c_1).$$

This expression is exactly the cofactor expansion of

$$\det \begin{pmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{pmatrix}$$

along the first row. Hence,

$$\boxed{\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \det \begin{pmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{pmatrix}.}$$

Cyclic Property.
The scalar triple product remains unchanged under cyclic permutation:

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \mathbf{b} \cdot (\mathbf{c} \times \mathbf{a}) = \mathbf{c} \cdot (\mathbf{a} \times \mathbf{b}).$$

Proof.
From the determinant form, we know that

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \det \begin{pmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{pmatrix}.$$

A determinant does not change under cyclic permutation of rows. Hence,

$$\det\begin{pmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{pmatrix} = \det\begin{pmatrix} b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \\ a_1 & a_2 & a_3 \end{pmatrix} = \det\begin{pmatrix} c_1 & c_2 & c_3 \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{pmatrix}.$$

Thus,

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \mathbf{b} \cdot (\mathbf{c} \times \mathbf{a}) = \mathbf{c} \cdot (\mathbf{a} \times \mathbf{b}).$$

If two vectors are interchanged (not cyclically permuted), the sign changes because swapping two rows of a determinant changes its sign:

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = -\mathbf{a} \cdot (\mathbf{c} \times \mathbf{b}).$$

Geometrical Meaning.
The quantity

$$|\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})|$$

represents the volume of the parallelepiped formed by $\mathbf{a}, \mathbf{b}, \mathbf{c}$.
Here,

$$\|\mathbf{b} \times \mathbf{c}\|$$

gives the area of the base parallelogram formed by $\mathbf{b}$ and $\mathbf{c}$, and taking the dot product with $\mathbf{a}$ extracts the height relative to that base.
Hence,

$$\text{Volume} = (\text{base area}) \times (\text{height}).$$

The sign of the scalar triple product determines the orientation (right-handed or left-handed system).

Coplanarity Condition.

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = 0 \quad \Longleftrightarrow \quad \mathbf{a}, \mathbf{b}, \mathbf{c} \text{ are coplanar.}$$

```python
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

# ====================================
# Define vectors
# ====================================
u = np.array([2, 1, 3])
v = np.array([1, 2, 0])
w = np.array([0, 1, 2])
```

```
origin = np.array([0, 0, 0])

print("u =", u)
print("v =", v)
print("w =", w)

# ====================================
# Cross Products
# ====================================
u_cross_v = np.cross(u, v)
v_cross_w = np.cross(v, w)

print("u x v =", u_cross_v)
print("v x w =", v_cross_w)

# ====================================
# Scalar Triple Products
# ====================================
triple1 = np.dot(u, v_cross_w)
triple2 = np.dot(u_cross_v, w)
triple3 = np.dot(v, np.cross(w, u))

print("u . (v x w) =", triple1)
print("(u x v) . w =", triple2)
print("v . (w x u) =", triple3)

volume = abs(triple1)

# ====================================
# Projections
# ====================================
if np.linalg.norm(v_cross_w) != 0:
    proj_u_on_vxw = (np.dot(u, v_cross_w) /
                     np.linalg.norm(v_cross_w)**2) * v_cross_w
else:
    proj_u_on_vxw = np.array([0, 0, 0])

if np.linalg.norm(u_cross_v) != 0:
    proj_w_on_uxv = (np.dot(w, u_cross_v) /
                     np.linalg.norm(u_cross_v)**2) * u_cross_v
else:
    proj_w_on_uxv = np.array([0, 0, 0])

print("Projection of u on v x w =", proj_u_on_vxw)
print("Projection of w on u x v =", proj_w_on_uxv)

# ====================================
```

```
# Areas
# ====================================
area_uv = np.linalg.norm(u_cross_v)
area_vw = np.linalg.norm(v_cross_w)

print("Area (u,v) =", area_uv)
print("Area (v,w) =", area_vw)

# ====================================
# Plot
# ====================================
fig = plt.figure(figsize=(8,8))
ax = fig.add_subplot(111, projection='3d')

ax.quiver(*origin, *u, linewidth=2)
ax.quiver(*origin, *v, linewidth=2)
ax.quiver(*origin, *w, linewidth=2)
ax.quiver(*origin, *u_cross_v, linestyle='dashed')
ax.quiver(*origin, *v_cross_w, linestyle='dashed')

ax.quiver(*origin, *proj_u_on_vxw,
          linestyle='dotted', linewidth=2)
ax.quiver(*origin, *proj_w_on_uxv,
          linestyle='dotted', linewidth=2)

ax.text(*u, "u")
ax.text(*v, "v")
ax.text(*w, "w")

base_vw = [origin, v, v + w, w]
ax.add_collection3d(
    Poly3DCollection([base_vw], alpha=0.3)
)

parallelepiped_faces = [
    [origin, v, v + w, w],
    [origin + u, v + u, v + w + u, w + u]
]

ax.add_collection3d(
    Poly3DCollection(parallelepiped_faces, alpha=0.1)
)

ax.text2D(0.05, 0.95,
          f"u . (v x w) = {triple1}",
          transform=ax.transAxes)
```

```
ax.text2D(0.05, 0.91,
          f"Volume = {volume:.2f}",
          transform=ax.transAxes)

plt.show()
```

Problem: Find the scalar triple product of the vectors

$$\mathbf{u} = (2, 1, 3), \quad \mathbf{v} = (1, 2, 0), \quad \mathbf{w} = (0, 1, 2).$$

OUTPUT

u = [2 1 3]
v = [1 2 0]
w = [0 1 2]
u × v = [-6 3 3]
v × w = [4 -2 1]
u · (v × w) = 9
(u × v) · w = 9
v · (w × u) = 9
Projection of u on v × w = [1.71428571 -0.85714286 0.42857143]
Projection of w on u × v = [-1. 0.5 0.5]
Area of parallelogram formed by u and v = 7.3484692283495345

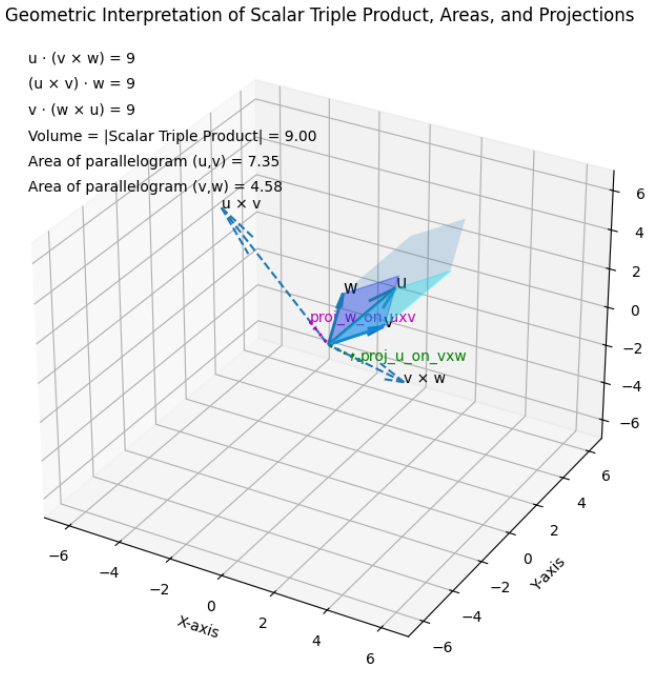

Figure 13:

6 Least Squares Approximation

Introduction.

In many practical situations, a function $f(x)$ cannot be represented exactly using a simpler function (such as a polynomial of low degree). In such cases, we seek the *best approximation* to f from a given finite-dimensional subspace.

The method of least squares provides a systematic way to find this best approximation by minimizing the error in the L^2 sense.

Inner Product Space of Functions

Let $C[a, b]$ denote the space of continuous functions on $[a, b]$. Define an inner product on this space by

$$\langle f, g\rangle = \int_a^b f(x)g(x)\,dx.$$

The induced norm is

$$\|f\| = \sqrt{\langle f, f\rangle} = \left(\int_a^b |f(x)|^2 dx\right)^{1/2}.$$

Thus $C[a, b]$ becomes an inner product space.

Problem of Approximation

Let W be a finite-dimensional subspace of $C[a, b]$ with basis

$$\{\phi_1, \phi_2, \ldots, \phi_n\}.$$

Given a function $f(x)$, we seek an approximation

$$p(x) = c_1\phi_1(x) + c_2\phi_2(x) + \cdots + c_n\phi_n(x)$$

such that the error

$$\|f - p\|$$

is minimized.

This is called the **least squares approximation** of f in W.

Orthogonality Condition

The best approximation $p \in W$ satisfies the condition

$$f - p \perp W.$$

That is,

$$\langle f - p, \phi_i \rangle = 0 \quad \text{for } i = 1, 2, \ldots, n.$$

This gives the system of equations

$$\int_a^b (f(x) - p(x))\phi_i(x)\, dx = 0, \quad i = 1, \ldots, n.$$

If $\{\phi_1, \ldots, \phi_n\}$ is orthonormal, then
The coefficients simplify to

$$c_i = \langle f, \phi_i \rangle.$$

Hence,

$$p(x) = \sum_{i=1}^{n} \langle f, \phi_i \rangle \phi_i(x).$$

Geometrical Interpretation

Least squares approximation is the orthogonal projection of f onto the subspace W.

The approximation p is the unique function in W such that

$$\|f - p\| \leq \|f - w\| \quad \text{for all } w \in W.$$

Thus, the error vector $f - p$ is orthogonal to the subspace W, just as in finite-dimensional Euclidean spaces.

```
import numpy as np
import matplotlib.pyplot as plt

# --- User Input for Function and Interval ---
function_str = input("Enter the function f(x) in terms of 'x'
    (e.g., 'np.sin(x)', 'x**2'): ")
start_limit = eval(input("Enter the start of the interval (e.g.,
    0): "))
end_limit = eval(input("Enter the end of the interval (e.g., np.pi
    for pi): "))
N_TERMS = int(input("Enter the number of approximations you need
    (e.g., 3 for g1, g2, g3; 20 for up to g20): "))
```

```
# Define the interval for x
x = np.linspace(start_limit, end_limit, 1000)

# Define the function f(x) using eval to interpret user input
try:
    y_func = eval(function_str)
except (NameError, SyntaxError) as e:
    print(f"Error evaluating function: {e}. Please ensure correct
       syntax and available numpy functions.")
    y_func = np.zeros_like(x) # Fallback to zeros
    function_str = "Error in function input"

print(f"\nFunction f(x) = {function_str} defined over
   [{start_limit}, {end_limit}].")

# 1. Define the initial basis functions (x^0, x^1, ...,
   x^(N_TERMS-1))
basis_functions_p = [x**i for i in range(N_TERMS)]

# 2. Calculate the differential dx
dx = x[1] - x[0]

# 3. Define the inner_product function
def inner_product(f, g, dx):
    return np.sum(f * g * dx)

# 4. Define the norm function
def norm(f, dx):
    return np.sqrt(inner_product(f, f, dx))

# 5. Apply the Gram-Schmidt process
u_vectors = []
w_vectors = []
norm_u_vals = [] # Store norms of u_vectors for polynomial
   reconstruction
ip_cache = {} # Store inner products for polynomial reconstruction,
   limited to first few needed

print(f"\nApplying Gram-Schmidt process for {N_TERMS} terms...")

for i in range(N_TERMS):
    p_i = basis_functions_p[i]

    u_i = p_i
    # Subtract projections onto previously orthonormalized vectors
    for j in range(i):
```

```
        w_j = w_vectors[j]
        proj_p_i_on_w_j = inner_product(p_i, w_j, dx) * w_j
        u_i = u_i - proj_p_i_on_w_j

        # Cache inner products needed for polynomial reconstruction
            of w1, w2, w3
        if i < 3 and j < 3: # Cache for first 3 terms
            ip_cache[f'p{i}_w{j}'] = inner_product(p_i, w_j, dx)

    u_vectors.append(u_i)
    norm_u = norm(u_i, dx)
    norm_u_vals.append(norm_u)

    # Handle potential division by zero if a basis vector is
        linearly dependent or norm is zero
    if norm_u < 1e-10: # A small threshold for numerical stability
        print(f"Warning: Norm of u_{i} is near zero. This term may
            be linearly dependent.")
        w_i = np.zeros_like(x)
    else:
        w_i = u_i / norm_u
    w_vectors.append(w_i)

print(f"Orthonormal basis {{w1, ..., w{N_TERMS}}} calculated using
    Gram-Schmidt process.")

# Calculate coefficients and approximations
coefficients_c = []
approximations_g = []
current_approximation_g = np.zeros_like(x)

print(f"\nCalculating {N_TERMS} successive approximations...")
for i in range(N_TERMS):
    w_i = w_vectors[i]
    c_i = inner_product(y_func, w_i, dx)
    coefficients_c.append(c_i)

    current_approximation_g = current_approximation_g + c_i * w_i
    approximations_g.append(current_approximation_g)

print(f"Coefficients c1 to c{N_TERMS} calculated.")
print(f"Approximations g1 to g{N_TERMS} calculated.")


# --- Display Selected Functional Forms ---
# We will display the first few orthonormal basis functions and
    approximations (up to 3 for brevity).
```

```
# This requires reconstructing their polynomial coefficients based
    on numerical inner products and norms.

print("\nFunctional forms of the first few orthonormal basis
    functions:")

if N_TERMS >= 1:
    k0 = 1 / norm_u_vals[0]
    w0_coeff_A = k0
    print(f"w1(x) = {w0_coeff_A:.6f}")

if N_TERMS >= 2:
    k1 = 1 / norm_u_vals[1]
    ip_p1_w0 = ip_cache.get('p1_w0',
        inner_product(basis_functions_p[1], w_vectors[0], dx))
    w1_coeff_B = k1
    w1_coeff_C = -ip_p1_w0 * w0_coeff_A * k1
    print(f"w2(x) = {w1_coeff_B:.6f}*x + {w1_coeff_C:.6f}")

if N_TERMS >= 3:
    k2 = 1 / norm_u_vals[2]
    ip_p2_w0 = ip_cache.get('p2_w0',
        inner_product(basis_functions_p[2], w_vectors[0], dx))
    ip_p2_w1 = ip_cache.get('p2_w1',
        inner_product(basis_functions_p[2], w_vectors[1], dx))

    w2_coeff_D = k2
    w2_coeff_E = k2 * (-ip_p2_w1 * w1_coeff_B)
    w2_coeff_F = k2 * (-ip_p2_w0 * w0_coeff_A - ip_p2_w1 *
        w1_coeff_C)
    print(f"w3(x) = {w2_coeff_D:.6f}*x^2 + {w2_coeff_E:.6f}*x +
        {w2_coeff_F:.6f}")

if N_TERMS > 3:
    print(f"Functional forms for higher-order terms (w4 to
        w{N_TERMS}) would involve increasingly complex polynomials.")


print("\nFunctional forms of the first few approximations (as
    polynomials in x):")

if N_TERMS >= 1:
    print(f"g1(x) = {coefficients_c[0] * w0_coeff_A:.6f}")

if N_TERMS >= 2:
    g1_coeff_x = coefficients_c[1] * w1_coeff_B
    g1_coeff_const = coefficients_c[0] * w0_coeff_A +
```

```
        coefficients_c[1] * w1_coeff_C
    print(f"g2(x) = {g1_coeff_x:.6f}*x + {g1_coeff_const:.6f}")

if N_TERMS >= 3:
    g2_coeff_x2 = coefficients_c[2] * w2_coeff_D
    g2_coeff_x = coefficients_c[1] * w1_coeff_B + coefficients_c[2]
        * w2_coeff_E
    g2_coeff_const = coefficients_c[0] * w0_coeff_A +
        coefficients_c[1] * w1_coeff_C + coefficients_c[2] *
        w2_coeff_F
    print(f"g3(x) = {g2_coeff_x2:.6f}*x^2 + {g2_coeff_x:.6f}*x +
        {g2_coeff_const:.6f}")

if N_TERMS > 3:
    print(f"Functional forms for higher-order approximations (g4 to
        g{N_TERMS}) would be increasingly complex polynomials.")


# --- Plot Selected Approximations ---
plt.figure(figsize=(12, 8))

plt.plot(x, y_func, label=f'f(x) = {function_str}', color='blue',
    linewidth=2)

# Define plot indices for selected approximations dynamically
plot_indices = []

# Always plot g1, g2, g3 if available
if N_TERMS >= 1:
    plot_indices.append(0) # g1
if N_TERMS >= 2:
    plot_indices.append(1) # g2
if N_TERMS >= 3:
    plot_indices.append(2) # g3

# Add intermediate and final approximations if N_TERMS is large
    enough
if N_TERMS > 3:
    # Add g5, g10, g15, g20 if N_TERMS allows
    if N_TERMS >= 5 and 4 not in plot_indices:
        plot_indices.append(4) # g5
    if N_TERMS >= 10 and 9 not in plot_indices:
        plot_indices.append(9) # g10
    if N_TERMS >= 15 and 14 not in plot_indices:
        plot_indices.append(14) # g15
    if N_TERMS >= 20 and 19 not in plot_indices:
        plot_indices.append(19) # g20
```

```
    # Always add the very last approximation if it's not already
       included
    if (N_TERMS - 1) not in plot_indices and N_TERMS > 0:
        plot_indices.append(N_TERMS - 1)

# Ensure unique and sorted indices
plot_indices = sorted(list(set(plot_indices)))

colors = ['red', 'green', 'purple', 'orange', 'brown', 'cyan',
   'magenta'] # Added more colors for more plots
linestyles = ['--', ':', '-.', ':', '--', '-', 'dotted'] # Added
   more linestyles

for i, idx in enumerate(plot_indices):
    if idx < len(approximations_g):
        plt.plot(x, approximations_g[idx], label=f'g{idx+1}(x)
           ({idx+1}th approximation)',
                 color=colors[i % len(colors)],
                 linestyle=linestyles[i % len(linestyles)],
                 linewidth=1.5)

plt.title(f'Function Approximation of {function_str} with {N_TERMS}
   Orthonormal Basis Vectors')
plt.xlabel('x')
plt.ylabel('f(x) / g(x)')
plt.legend()
plt.grid(True)
plt.show()
```

Problem: Approximate

$$f(x) = e^x, \quad 0 \le x \le 1$$

by a linear function

$$g(x) = a_0 + a_1 x.$$

OUTPUT

Enter the function f(x) in terms of 'x' (e.g., 'np.sin(x)', 'x**2'): np.e**x
Enter the start of the interval (e.g., 0): 0
Enter the end of the interval (e.g., np.pi for pi): 1
Enter the number of approximations you need (e.g., 3 for g1, g2, g3; 20 for up to g20): 2

Function f(x) = np.e**x defined over [0, 1].

Applying Gram-Schmidt process for 2 terms...
Orthonormal basis w1, ..., w2 calculated using Gram-Schmidt process.

Calculating 2 successive approximations...
Coefficients c1 to c2 calculated.
Approximations g1 to g2 calculated.

Functional forms of the first few orthonormal basis functions:
w1(x) = 0.999500
w2(x) = 3.458908*x + -1.729454

Functional forms of the first few approximations (as polynomials in x):
g1(x) = 1.718423
g2(x) = 1.690393*x + 0.873226

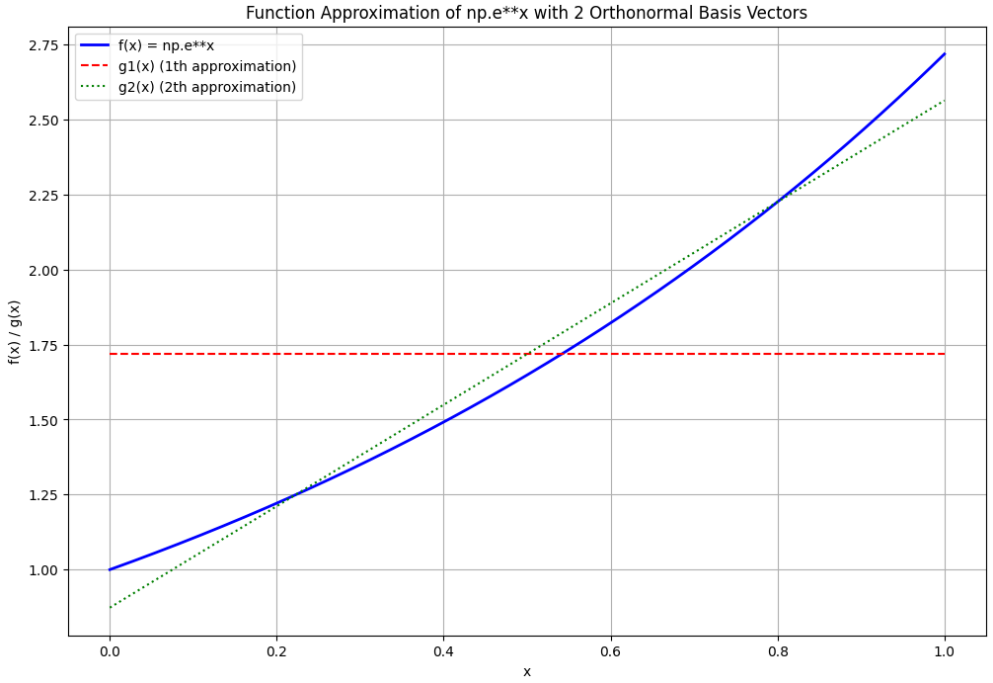

Figure 14:

Problem: Approximate $f(x) = e^x$ on $[0, 1]$ by

$$g(x) = a_0 + a_1 x + a_2 x^2.$$

OUTPUT

Enter the function f(x) in terms of 'x' (e.g., 'np.sin(x)', 'x**2'): np.e**x
Enter the start of the interval (e.g., 0): 0
Enter the end of the interval (e.g., np.pi for pi): 1

Enter the number of approximations you need (e.g., 3 for g1, g2, g3; 20 for up to g20): 3

Function f(x) = np.e**x defined over [0, 1].

Applying Gram-Schmidt process for 3 terms...
Orthonormal basis w1, ..., w3 calculated using Gram-Schmidt process.

Calculating 3 successive approximations...
Coefficients c1 to c3 calculated.
Approximations g1 to g3 calculated.

Functional forms of the first few orthonormal basis functions:
w1(x) = 0.999500
w2(x) = 3.458908*x + -1.729454
$w3(x) = 13.382925 * x^2 + -13.382925 * x + 2.228255$

Functional forms of the first few approximations (as polynomials in x):
g1(x) = 1.718423
g2(x) = 1.690393*x + 0.873226
$g3(x) = 0.839214 * x^2 + 0.851179 * x + 1.012955$

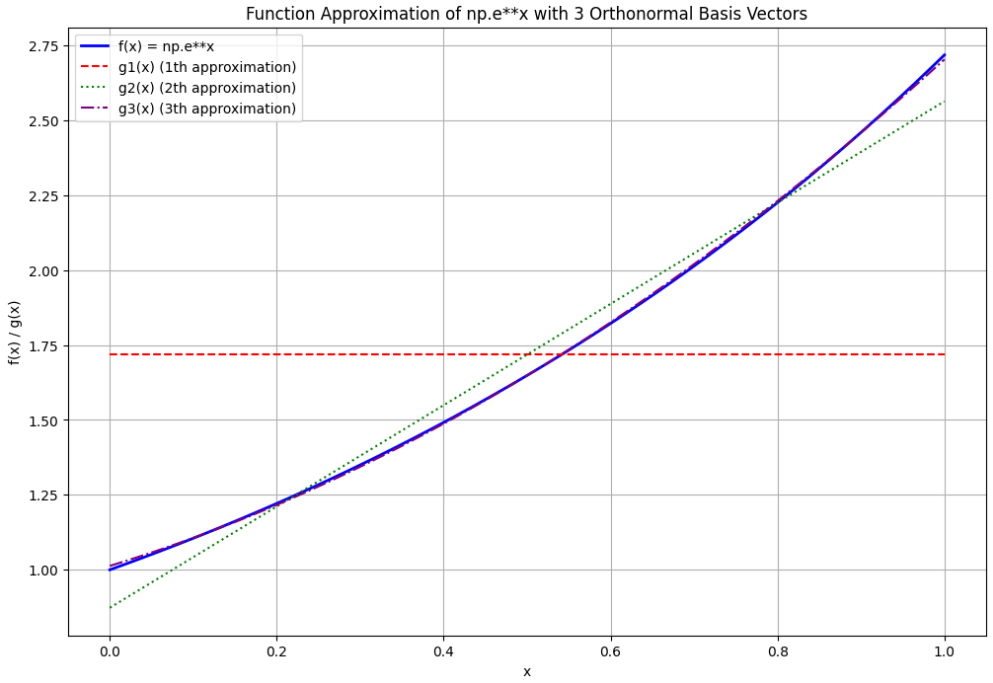

Figure 15:

Problem: Approximate $f(x) = sin(x)$ on $[0, \pi]$ by

$$g(x) = a_0 + a_1 x + a_2 x^2.$$

OUTPUT

Enter the function f(x) in terms of 'x' (e.g., 'np.sin(x)', 'x**2'): np.sin(x)
Enter the start of the interval (e.g., 0): 0
Enter the end of the interval (e.g., np.pi for pi): np.pi
Enter the number of approximations you need (e.g., 3 for g1, g2, g3; 20 for up to g20): 3

Function f(x) = np.sin(x) defined over [0, 3.141592653589793].

Applying Gram-Schmidt process for 3 terms...
Orthonormal basis w1, ..., w3 calculated using Gram-Schmidt process.

Calculating 3 successive approximations...
Coefficients c1 to c3 calculated.
Approximations g1 to g3 calculated.

Functional forms of the first few orthonormal basis functions:
w1(x) = 0.563907
w2(x) = 0.621175*x + -0.975740
$w3(x) = 0.765026 * x^2 + -2.403401 * x + 1.257158$

Functional forms of the first few approximations (as polynomials in x):
g1(x) = 0.635983
g2(x) = 0.000000*x + 0.635983
$g3(x) = -0.417544 * x^2 + 1.311755 * x + -0.050163$

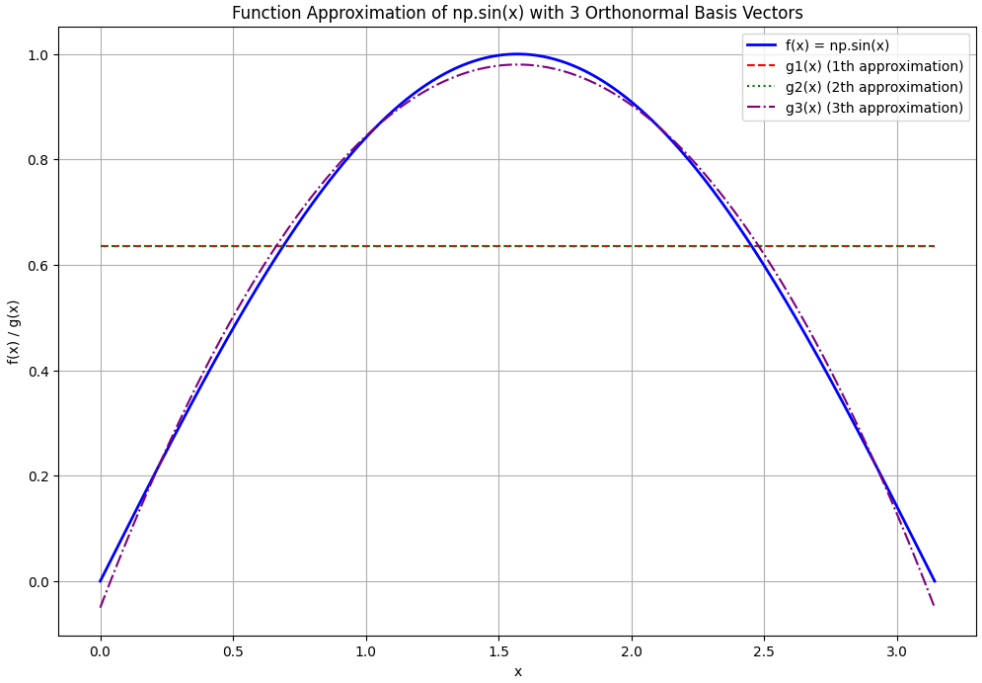

Figure 16:

Problem: Approximate the function

$$f(x) = e^x + \sin x + 5$$

on the interval

$$[0, 4\pi]$$

using 20 approximations.

OUTPUT

Enter the function f(x) in terms of 'x' (e.g., 'np.sin(x)', 'x**2'): np.e**x+np.sin(x)+5
Enter the start of the interval (e.g., 0): 0
Enter the end of the interval (e.g., np.pi for pi): 4*np.pi
Enter the number of approximations you need (e.g., 3 for g1, g2, g3; 20 for up to g20): 20

Function f(x) = np.e**x+np.sin(x)+5 defined over [0, 12.566370614359172].

Applying Gram-Schmidt process for 20 terms...
Orthonormal basis w1, ..., w20 calculated using Gram-Schmidt process.

Calculating 20 successive approximations...
Coefficients c1 to c20 calculated.
Approximations g1 to g20 calculated.

Functional forms of the first few orthonormal basis functions:
$w1(x) = 0.281954$
$w2(x) = 0.077647 * x + -0.487870$
$w3(x) = 0.023907 * x^2 + -0.300425 * x + 0.628579$
Functional forms for higher-order terms ($w4$ to $w20$) would involve increasingly complex polynomials.

Functional forms of the first few approximations (as polynomials in x):
$g1(x) = 22944.722760$
$g2(x) = 9202.218544 * x + -34874.521587$
$g3(x) = 2604.568884 * x^2 + -23527.759343 * x + 33606.365677$
Functional forms for higher-order approximations ($g4$ to $g20$) would be increasingly complex polynomials.

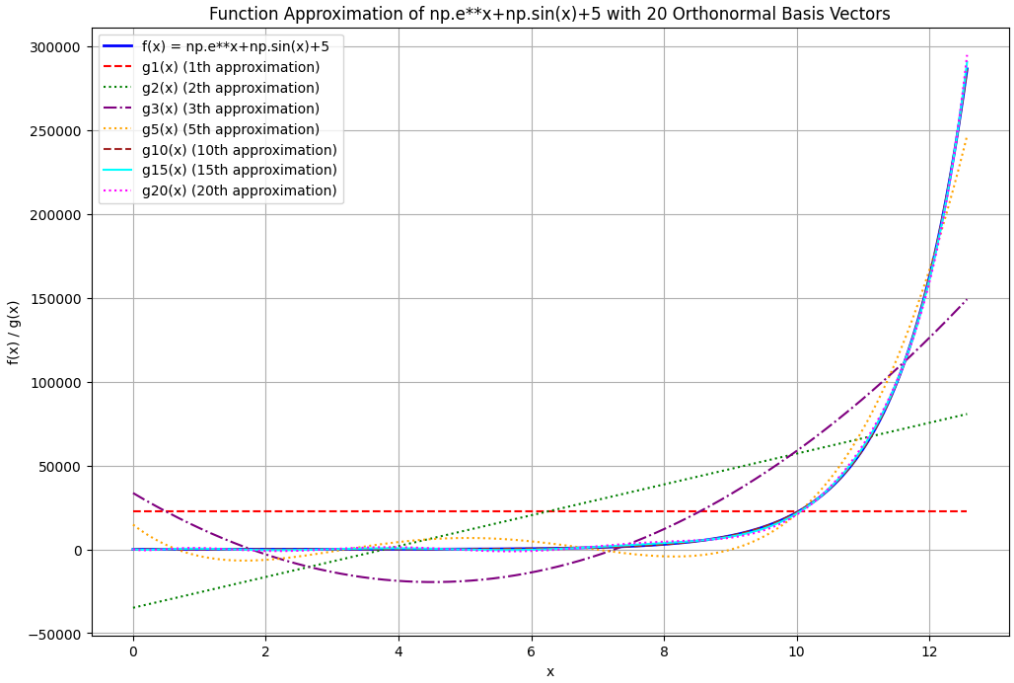

Figure 17:

6.1 Fourier Approximation

Introduction.

Fourier approximation is a special case of least squares approximation in which a function is approximated using trigonometric functions. It is based on the fact that sine and cosine functions form an orthogonal system on a suitable interval.

Inner Product on $[-\pi, \pi]$

Consider the space of functions defined on $[-\pi, \pi]$ with inner product

$$\langle f, g\rangle = \int_{-\pi}^{\pi} f(x)g(x)\,dx.$$

The corresponding norm is

$$\|f\| = \left(\int_{-\pi}^{\pi} |f(x)|^2 dx\right)^{1/2}.$$

Orthogonal Trigonometric System

The functions

$$\{1, \cos nx, \sin nx\}_{n=1}^{\infty}$$

form an orthogonal set on $[-\pi, \pi]$. In particular,

$$\int_{-\pi}^{\pi} \cos(mx)\cos(nx)\,dx = 0 \quad (m \neq n),$$

$$\int_{-\pi}^{\pi} \sin(mx)\sin(nx)\,dx = 0 \quad (m \neq n),$$

$$\int_{-\pi}^{\pi} \sin(mx)\cos(nx)\,dx = 0.$$

Fourier Series

Let $f(x)$ be a piecewise continuous function on $[-\pi, \pi]$. Its Fourier series is given by

$$f(x) \sim \frac{a_0}{2} + \sum_{n=1}^{\infty} \left(a_n \cos nx + b_n \sin nx\right),$$

where the coefficients are

$$a_n = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\cos(nx)\,dx,$$

$$b_n = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\sin(nx)\,dx.$$

Least Squares Interpretation

Let W_n be the subspace spanned by

$$\{1, \cos x, \sin x, \ldots, \cos nx, \sin nx\}.$$

The nth partial sum

$$S_n(x) = \frac{a_0}{2} + \sum_{k=1}^{n} (a_k \cos kx + b_k \sin kx)$$

is the orthogonal projection of f onto W_n.
Thus, S_n minimizes the error

$$\|f - p\|$$

among all trigonometric polynomials $p \in W_n$.

Geometrical Meaning

Fourier approximation is the projection of a function onto a subspace generated by orthogonal trigonometric functions.

The error function

$$f - S_n$$

is orthogonal to every function in W_n, i.e.,

$$\langle f - S_n, \phi \rangle = 0 \quad \text{for all } \phi \in W_n.$$

Hence Fourier series generalizes the idea of orthogonal projection from finite-dimensional vector spaces to infinite-dimensional function spaces.

Remark.

If the trigonometric system is normalized to be orthonormal, then the Fourier coefficients are simply the inner products of f with the basis functions.

```
import sympy
import numpy as np
import matplotlib.pyplot as plt

def get_user_inputs():
    """Prompts the user for the function, interval limits, and
       number of Fourier approximations."""
    while True:
        f_str = input("Enter the function f(x) (e.g., 'sin(x)',
           'x**2', 'exp(x)'): ")
        try:
```

```
            x_temp = sympy.Symbol('x') # Use a temporary symbol for
                parsing check
            # Replace 'np.pi' with 'pi' for symbolic processing and
                remove 'np' from locals
            sympy.sympify(f_str.replace('np.pi', 'pi'),
                locals={'x': x_temp, 'sin': sympy.sin, 'cos':
                sympy.cos, 'exp': sympy.exp, 'pi': sympy.pi})
            break
        except (sympy.SympifyError, SyntaxError) as e:
            print(f"Invalid function expression: {e}. Please try
                again.")

    while True:
        try:
            # Allow sympy expressions like 'np.pi' for limits, as
                these are converted to floats.
            a_str = input("Enter the lower limit 'a' of the
                interval: ")
            a = float(sympy.sympify(a_str, locals={'pi': sympy.pi,
                'np': np}))
            break
        except (ValueError, sympy.SympifyError) as e:
            print(f"Invalid input for 'a': {e}. Please enter a
                numerical value or sympy expression.")

    while True:
        try:
            # Allow sympy expressions like 'np.pi' for limits, as
                these are converted to floats.
            b_str = input("Enter the upper limit 'b' of the
                interval: ")
            b = float(sympy.sympify(b_str, locals={'pi': sympy.pi,
                'np': np}))
            if b <= a:
                print("Upper limit 'b' must be greater than lower
                    limit 'a'. Please try again.")
            else:
                break
        except (ValueError, sympy.SympifyError) as e:
            print(f"Invalid input for 'b': {e}. Please enter a
                numerical value or sympy expression.")

    while True:
        try:
            N = int(input("Enter the number of Fourier
                approximations 'N': "))
            if N <= 0:
```

```
                print("The number of approximations 'N' must be a
                    positive integer. Please try again.")
            else:
                break
        except ValueError:
            print("Invalid input. Please enter an integer value for
                'N'.")

    return f_str, a, b, N

def get_orthonormal_fourier_basis_function(k, x_sym, a, b):
    """Calculates the k-th orthonormal Fourier basis function over
        [a, b]."""
    L = b - a

    if k == 1:
        return sympy.Rational(1, sympy.sqrt(L))
    elif k % 2 == 0:  # Even k corresponds to cosine terms
        n = k / 2
        return sympy.sqrt(sympy.Rational(2, L)) * sympy.cos(2 *
            sympy.pi * n * x_sym / L)
    else:  # Odd k > 1 corresponds to sine terms
        n = (k - 1) / 2
        return sympy.sqrt(sympy.Rational(2, L)) * sympy.sin(2 *
            sympy.pi * n * x_sym / L)

def calculate_fourier_coefficient(f_expr, w_k_expr, x_sym, a, b):
    """Calculates the Fourier coefficient (inner product) <f, w_k>
        = integral from a to b of f(x) * w_k(x) dx."""
    integrand = f_expr * w_k_expr
    coefficient = sympy.integrate(integrand, (x_sym, a, b))
    return coefficient

def construct_fourier_approximation(coefficients, x_sym, a, b,
    num_terms):
    """Constructs the Fourier series approximation up to
        num_terms."""
    fourier_approx = 0
    for i in range(num_terms):
        if i < len(coefficients):
            fourier_approx += coefficients[i] *
                get_orthonormal_fourier_basis_function(i + 1, x_sym,
                a, b)
        else:
            break
    return fourier_approx
```

```
# --- Main execution flow ---

# 1. Get User Inputs
f_string, lower_limit, upper_limit, num_approximations =
    get_user_inputs()
print(f"\nUser Inputs:\nFunction f(x): {f_string}\nLower limit (a):
    {lower_limit}\nUpper limit (b): {upper_limit}\nNumber of
    approximations (N): {num_approximations}")

# Define the symbolic variable
x = sympy.Symbol('x')

# Parse the user-defined function string into a sympy expression
# Replace 'np.pi' with 'pi' for symbolic processing and remove 'np'
    from locals
f_expr = sympy.sympify(f_string.replace('np.pi', 'pi'),
    locals={'x': x, 'sin': sympy.sin, 'cos': sympy.cos, 'exp':
    sympy.exp, 'pi': sympy.pi})

# 2. Calculate Fourier Coefficients
fourier_coefficients = []
print("\nCalculating Fourier Coefficients...")
for k in range(1, num_approximations + 1):
    w_k_expr = get_orthonormal_fourier_basis_function(k, x,
        lower_limit, upper_limit)
    c_k = calculate_fourier_coefficient(f_expr, w_k_expr, x,
        lower_limit, upper_limit)
    fourier_coefficients.append(c_k)
    # print(f"c_{k} = {c_k}") # Uncomment to see individual
        coefficients
print(f"Calculated {len(fourier_coefficients)} Fourier
    coefficients.")

# 3. Construct Fourier Approximations
fourier_approximations = []
print("\nConstructing Fourier Approximations...")
for n_terms in range(1, num_approximations + 1):
    approx_n =
        construct_fourier_approximation(fourier_coefficients, x,
        lower_limit, upper_limit, n_terms)
    fourier_approximations.append(approx_n)
    # print(f"Fourier Approximation (N={n_terms}): {approx_n}") #
        Uncomment to see individual approximations
print(f"Constructed {len(fourier_approximations)} Fourier series
    approximations.")

# 4. Prepare for Numerical Evaluation (Lambdify)
```

```
f_numeric = sympy.lambdify(x, f_expr, 'numpy')
fourier_approximations_numeric = []
for approx in fourier_approximations:
    fourier_approximations_numeric.append(sympy.lambdify(x, approx,
       'numpy'))
print("Original function and Fourier approximations have been
   lambdified for numerical evaluation.")

# 5. Plot Fourier Series Approximations
x_vals = np.linspace(lower_limit, upper_limit, 500)
y_f = f_numeric(x_vals)

# Evaluate each fourier_approximations_numeric function and ensure
   consistent shape
y_approximations = []
for approx_func in fourier_approximations_numeric:
    y_val = approx_func(x_vals)
    # If y_val is a scalar, convert it to an array of the same
       shape as x_vals
    if np.isscalar(y_val):
        y_approximations.append(np.full_like(x_vals, y_val))
    else:
        y_approximations.append(y_val)

plt.figure(figsize=(12, 8))
plt.plot(x_vals, y_f, label=f'f(x) = {f_string}', color='blue',
   linewidth=2)

colors = plt.cm.viridis(np.linspace(0, 1, num_approximations + 1))
for i, y_approx in enumerate(y_approximations):
    plt.plot(x_vals, y_approx, label=f'N={i+1} Approximation',
       linestyle='--', color=colors[i+1])

plt.title(f'Fourier Series Approximations for f(x) = {f_string} on
   [{lower_limit}, {upper_limit}]', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('f(x) / Approximation', fontsize=12)
# Adjust legend to be outside the plot area
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left',
   borderaxespad=0., fontsize=10)
plt.grid(True)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.tight_layout() # Adjust layout to prevent labels/legend from
   being cut off
plt.show()
```

Problem: Using Fourier approximation, approximate the function

$$f(x) = 1.5\sin(2\pi x) + 0.8\sin(6\pi x) + 0.5\sin(10\pi x) + 0.3\sin(16\pi x)$$

on the interval

$$[0, 1]$$

using 20 approximations.

OUTPUT

Enter the function f(x) (e.g., 'sin(x)', 'x**2', 'exp(x)'): 1.5*sin(2*np.pi*x)+0.8*sin(6*np.pi*x)+
0.5*sin(10*np.pi*x)+0.3*sin(16*np.pi*x)
Enter the lower limit 'a' of the interval: 0
Enter the upper limit 'b' of the interval: 1
Enter the number of Fourier approximations 'N': 20

User Inputs:
Function f(x): 1.5*sin(2*np.pi*x)+0.8*sin(6*np.pi*x)+0.5*sin(10*np.pi*x)+0.3*sin(16*np.pi*x)
Lower limit (a): 0.0
Upper limit (b): 1.0
Number of approximations (N): 20

Calculating Fourier Coefficients...
Calculated 20 Fourier coefficients.

Constructing Fourier Approximations...
Constructed 20 Fourier series approximations.
Original function and Fourier approximations have been lambdified for numerical evaluation.

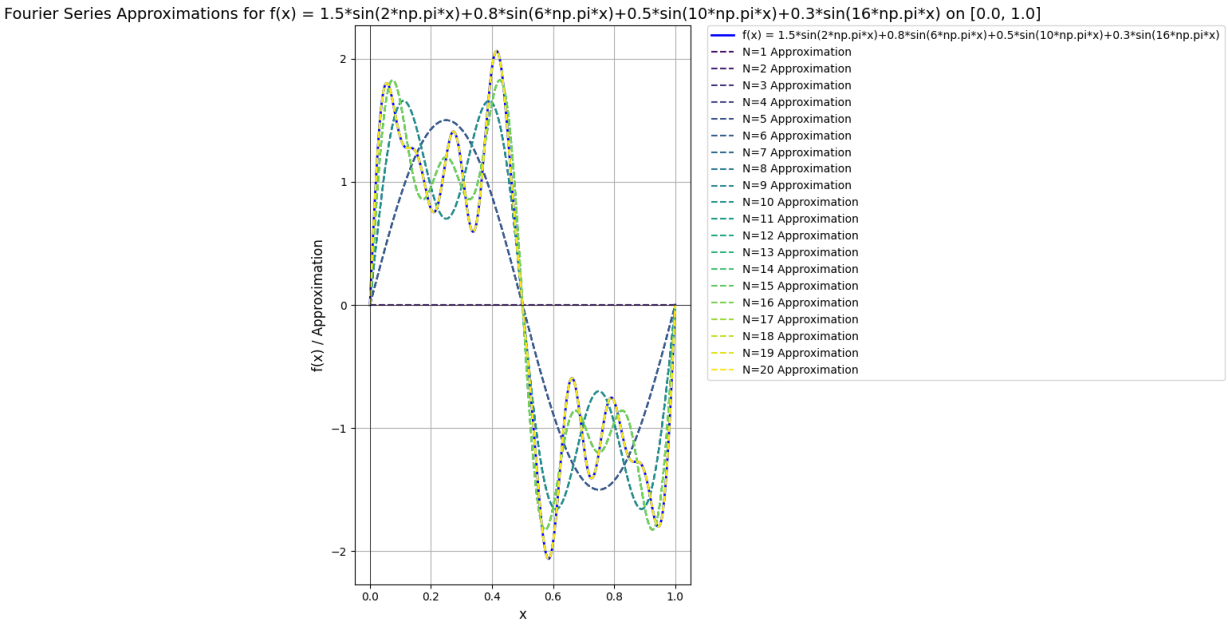

Figure 18:

Conclusion

In this module, we explored several important concepts in linear algebra and their applications. The Gram-Schmidt orthonormalisation process provides a systematic method to convert a set of linearly independent vectors into an orthonormal basis, simplifying computations in vector spaces.The study of orthonormal bases and projections of vectors onto subspaces helped us understand geometric interpretations of vector decomposition and we also examined the fundamental subspaces associated with matrices.The least squares method provided an efficient technique for obtaining approximate solutions to inconsistent systems, while cross product and vector operations helped in geometric interpretations of area and volume.Finally,the introduction to Fourier methods highlighted the importance of representing functions using orthogonal basis functions, which has wide applications in signal processing and numerical analysis.